ECE2132 Data Structures & Algorithms Lab — Start here

Built from the 9 manual files you uploaded (Exp 1–8, 10–12; AY 2026-27). Every program in this pack was compiled with g++ in strict mode and run on the manual's own sample inputs. The output boxes are copied from those real runs.

C++ from zero → Pointers (1–2) → Arrays (3–4) → Matrices (5–6) → Sorting (7–8) → Stack (9) → Queue (10) → Linked lists (11–12) → Exam day

What the lab exam asks of you

You will be given one experiment (possibly with a small change, like the manual's ASSIGNMENT lines). To score, you need four things:

1. **Write the record part**: aim, short algorithm, time/space complexity, result.
2. **Type the program** from memory, compile it, and fix your own typos from the compiler's messages.
3. **Run it** on a sample input and show the right output.
4. **Answer viva questions**: why this line, what happens if…, what is the complexity.

Each experiment chapter gives you those four, in that order.

How to use each chapter

> **THE ONE RULE**
>
> **Guess before you read.** Each chapter opens with a question box. Answer it in your head or on paper *before* reading on, even if you're unsure. A wrong guess still helps: the right answer sticks better after you've tried. Then type the program yourself, without looking. Re-reading feels like studying but doesn't make you able to type it from scratch.

- **Code boxes** (light grey) contain the corrected program. Type it yourself; don't copy-paste.
- **Screen boxes** (black) show what the program printed in a real run, with the typed input included.
- **Red boxes** show real compiler errors from the manual's broken versions, so you learn to read them.
- **Check** questions close each chapter. Answers are at the end of that chapter.
- **"If the examiner changes the question"** sections hold the manual's ASSIGNMENT variants. Each one is tested.

Manual errors found (and fixed in this pack)

The manual's listings are images, and many experiments print the code twice (a PROGRAM listing and an IMPLEMENTATION screenshot) that **don't match**. Everything below was checked by compiling and running it.

Exp	Problem in the manual	Effect	Fix in this pack
2	RESULT paragraph copied from Exp 1	Wrong result statement	Own result written
3	No check on position or free space	Position 0 or 50 corrupts memory	Position 1…n+1 checked
4	Every option asks "Enter value:" and reads a number it never uses (listing only; the screenshot is right)	With the manual's own sample input, the program waits forever for a value	Prompt removed; deleted element printed
4	Algorithm step 5 says "insertion menu"	Text slip	—
5	Algorithm step 7 is a stray deletion line; explanation table rows are shifted by one	Text slips	Correct table in chapter
6	Fig. 3 caption "A[i][j] to T[i][j]"	Should be T[j][i]	—
7, 8	`int arr[n];` with `n` read at run time	Not standard C++. Visual Studio refuses it; g++ only allows it as an extension	`int arr[100];` + range check
8	Aim and explanation promise Bubble, Selection, Insertion, `copyArray()` and `clock()` timing, but the code has only Quick Sort	Half the experiment is missing	Full comparison program added
8	Heading "Characteristics of Insertion Sort" over quick sort points	Text slip	—
10	Figure captions say "…Operation on **Stack**"	Copied from a stack experiment	—
10	Screenshot copy: `enqueue(value);` outside its `if`, choice 3 prints "Program terminated", bare `else`	Doesn't compile	Listing version used, braces fixed
10	Array named `queue`	Clashes with `std::queue` if `<bits/stdc++.h>` is included	Renamed `queueArr`
11	Screenshot copy: traverse never moves `temp` (infinite loop), insert never links the node, menu text corrupted	Broken	Listing version used (it is correct)
12	Listing: `insert(intvalue)`, menu strings without closing quotes, `delete temp;` outside the block where `temp` exists, `main` cut off	Doesn't compile	Fixed and completed

> **EXPERIMENT 9 IS MISSING**
>
> No "Exp 9" file was uploaded. Exp 10's figure captions read "Enqueue Operation on Stack", so Exp 9 is almost certainly **Stack using arrays**. Chapter 8 of this pack covers it, written in the same style as the Queue experiment. It is **not** from your manual; check with a classmate whether Exp 9 really is Stack.

The compiler on your laptop

To practise at home you need a C++ compiler. One is now installed on this laptop: **g++ 16.2 (WinLibs MinGW)**. Chapter 1 shows how to use it. In the lab you'll use whichever editor is there (VS Code, Code::Blocks or Dev-C++). The programs in this pack run the same on all three.

1 · C++ from zero (just what the lab needs)

No C assumed. Only the pieces the 12 experiments actually use. New pieces (pointers, 2-D arrays, struct, new/delete) are taught in the experiment that first needs them.

Run a program → Program skeleton → Variables + cin/cout → if / else → Loops → Arrays → Functions → Reading errors

1.1 Running a program

A C++ program is a text file ending in **.cpp**. A **compiler** (g++) turns it into an **.exe** you can run. If the text breaks the language rules, the compiler refuses and prints **errors** with line numbers. Nothing runs until every error is fixed.

Where	How to compile + run
Code::Blocks	File → New → Empty file → save as `exp1.cpp` → **F9** (Build and run)
Dev-C++	File → New → Source File → save as `.cpp` → **F11** (Compile & Run)
VS Code	Open the folder, then in the terminal (Ctrl+`): `g++ exp1.cpp -o exp1` then `.\exp1`
This laptop	Any new terminal: `g++ -static exp1.cpp -o exp1` then `.\exp1`

SAVE AS .CPP, NOT .C

A `.c` file is compiled as C, and `cout`, `cin`, `new` and `bool &` all fail. On this laptop, always add `-static`: another program on the PATH carries an older C++ library, and without `-static` your `.exe` may exit silently.

1.2 The skeleton every experiment starts with

GUESS FIRST

Which line would you guess makes the program *start*, and what do you think `return 0;` reports, and to whom?

P01_HELLO.CPP

```cpp
#include <iostream>          // lets us use cout and cin
using namespace std;         // so we can write cout instead of std::cout

int main()                   // the program starts running here
{
    cout << "Hello, lab!" << endl;    // print, then go to a new line
    return 0;                         // 0 = "finished normally"
}
```

SCREEN (REAL RUN)

```
Hello, lab!
```

- `#include <iostream>` brings in **cout** (print) and **cin** (read). Every experiment has it.
- `using namespace std;` lets you write `cout` instead of `std::cout`. Every experiment has it.
- `int main() { ... }` is where running starts. Statements run **top to bottom**.
- Every statement ends with `;`. Blocks of statements live inside `{ }`.
- `endl` (or `"\n"` inside a string) moves to a new line.
- `return 0;` tells the operating system "finished normally".
- `//` starts a comment. The compiler ignores the rest of the line.

1.3 Variables, input, arithmetic, decisions

A **variable** is a named box that holds one value of a fixed **type**:

Type	Holds	Example
`int`	whole numbers	`int n = 5;`
`float`	numbers with decimals	`float x = 2.5;`
`char`	one character	`char c = 'A';`
`bool`	`true` / `false`	`bool done = false;`

P02_INPUT.CPP

```cpp
#include <iostream>
using namespace std;

int main()
{
    int a, b;                          // two whole-number boxes
    cout << "Enter two numbers: ";
    cin >> a >> b;                     // read two numbers typed by the user

    int sum = a + b;
    cout << "Sum = " << sum << endl;
    cout << "a / b = " << a / b << endl;      // int / int drops the fraction
    cout << "a % b = " << a % b << endl;      // % = remainder

    if (a > b)
        cout << "a is bigger" << endl;
    else if (a == b)                   // == compares, = stores
        cout << "equal" << endl;
    else
        cout << "b is bigger" << endl;

    return 0;
}
```

SCREEN (REAL RUN)

```
Enter two numbers: 17 5
Sum = 22
a / b = 3
a % b = 2
a is bigger
```

THE THREE CLASSIC BEGINNER SLIPS

1. `=` **stores**, `==` **compares**. `if (a = b)` compiles but is wrong.
2. `int / int` **throws away** the fraction: 17 / 5 = 3. `%` gives the remainder: 17 % 5 = 2.
3. `cin >> a >> b;` uses `>>` (data flows *into* a). `cout << x;` uses `<<` (data flows *out to* the screen).

`if (…) {…} else if (…) {…} else {…}`: only the first true branch runs. With a single statement the `{ }` are optional; with more than one they are **required**. Exp 10's manual screenshot breaks exactly this rule.

1.4 Loops and arrays

An **array** is a row of boxes of the same type, numbered from **0**. `int arr[5]` has `arr[0]` … `arr[4]`. There is **no** `arr[5]`; using it reads or writes someone else's memory, and C++ won't stop you.

P03_LOOPS_ARRAYS.CPP

```cpp
#include <iostream>
using namespace std;

int main()
{
    int arr[5] = {10, 20, 30, 40, 50};       // indexes 0..4

    cout << "for loop: ";
    for (int i = 0; i < 5; i++)              // start; keep going while true; step
        cout << arr[i] << " ";
    cout << endl;

    cout << "backwards with while: ";
    int i = 4;
    while (i >= 0)
    {
        cout << arr[i] << " ";
        i--;
    }
    cout << endl;

    int choice;
    do                                       // body runs at least once
    {
        cout << "Enter 0 to stop: ";
        cin >> choice;
    } while (choice != 0);

    return 0;
}
```

SCREEN (REAL RUN)

```
for loop: 10 20 30 40 50
backwards with while: 50 40 30 20 10
Enter 0 to stop: 4
Enter 0 to stop: 0
```

Loop	Use it when	In the lab
`for (int i = 0; i < n; i++)`	you know how many times	walking an array (every experiment)
`while (condition)`	you stop on a condition	insertion sort's shifting, linked-list walking
`do { } while (condition);`	body must run **at least once**	menus (Exp 10–12), circular list traversal

`i++` means "add 1 to i"; `i--` means "subtract 1".

> **CHECK 1.1**
> What does this print? `for (int i = 4; i > 0; i--) cout << i;`

1.5 Functions

A **function** is a named piece of code you can call. From Exp 5 on, every experiment splits its work into functions (`inputMatrix` , `insertionSort` , `enqueue` , ...).

P04_FUNCTIONS.CPP

```cpp
#include <iostream>
using namespace std;

int square(int x)                  // takes an int, gives back an int
{
    return x * x;
}

void printArray(int arr[], int n)  // void = gives nothing back
{
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
}

void doubleAll(int arr[], int n)   // arrays are NOT copied: changes are seen by main
{
    for (int i = 0; i < n; i++)
        arr[i] = arr[i] * 2;
}

void tryToChange(int x)            // plain int IS copied: main's variable is untouched
{
    x = 99;
    cout << "inside tryToChange: " << x << endl;
}

void reallyChange(int &x)          // & = reference: x IS main's variable
{
    x = 99;
}

int main()
{
    cout << "square(7) = " << square(7) << endl;

    int a[3] = {1, 2, 3};
```

```
    doubleAll(a, 3);
    printArray(a, 3);

    int v = 5;
    tryToChange(v);
    cout << "after tryToChange: " << v << endl;
    reallyChange(v);
    cout << "after reallyChange: " << v << endl;
    return 0;
}
```

SCREEN (REAL RUN)

```
square(7) = 49
2 4 6
inside tryToChange: 99
after tryToChange: 5
after reallyChange: 99
```

Read the result carefully. It explains three things you'll meet again:

- `int square(int x)`: the first `int` is what the function *gives back* via `return`; `void` means it gives nothing back.
- **An array parameter is not copied.** The function works on the caller's array, so `doubleAll` changed `a`. That's why `insertionSort(arr, n)` sorts `main`'s array.
- **A plain `int` parameter is a copy**, so `tryToChange` changed only its own copy. Add `&` (`int &x`) and the parameter *is* the caller's variable. Exp 8's `swapElements(int &a, int &b)` needs this.

> **ORDER RULE**
> A function must appear **above** the place that calls it. Write helper functions first and `main()` last, as every experiment in the manual does.

1.6 Reading compiler errors (exam-day survival)

The compiler reports `file:LINE:COLUMN: error: message`. **Fix the first error first**: one missing `;` can produce ten errors below it. Here is the manual's own Exp 12 listing, compiled:

G++ SAYS

```
m12_listing.cpp:62:17: error: missing terminating " character
   62 |     {   cout << "\n1. Insert
      |                 ^
m12_listing.cpp:13:6: error: variable or field 'insert' declared void
   13 | void insert(intvalue)
      |      ^~~~~~
m12_listing.cpp:13:13: error: 'intvalue' was not declared in this scope
   13 | void insert(intvalue)
      |             ^~~~~~~~
m12_listing.cpp:46:15: error: unused variable 'temp' [-Werror=unused-variable]
   46 |     {   Node* temp = head;
      |               ^~~~
m12_listing.cpp:55:16: error: 'temp' was not declared in this scope; did you mean 'tm'?
   55 |     }   delete temp;
      |                ^~~~
m12_listing.cpp:62:14: error: no match for 'operator<<' (operand types are 'std::ostream' {aka
'std::basic_ostream<char>'} and 'std::ostream' {aka 'std::basic_ostream<char>'})
   62 |     {   cout << "\n1. Insert
      |         ~~~~ ^~
```

Message	Usual cause
`expected ';' before …`	missing `;` on the line **above**
`'x' was not declared in this scope`	typo in a name, or a variable used outside the `{ }` it was made in
`missing terminating " character`	a string without its closing `"`
`expected '}' at end of input`	a `{` was never closed
`'else' without a previous 'if'`	a statement squeezed between `}` and `else`
`ISO C++ forbids variable length array`	`int arr[n]` with `n` read at run time; use `int arr[100]`

Answers

Check 1.1: `4321`. It starts at 4, prints while i > 0, and stops once i becomes 0.

2 · Exp 1 + 2 — Pointers and pointer arithmetic

Manual Exp 1: "implement Pointers". Exp 2: "implement Pointer Arithmetic". Both have no user input, so they are the shortest programs in the lab.

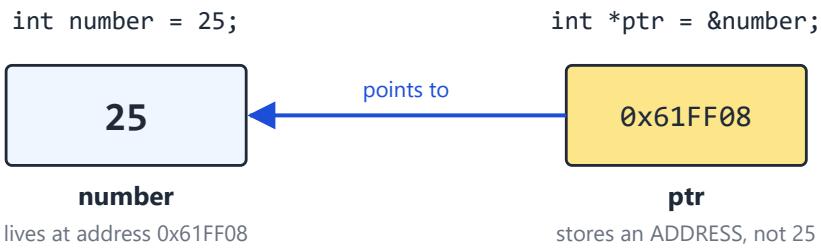
Address → Pointer variable → & and * → Pointer + 1 → Arrays are pointers → (later) linked lists

2.1 The idea

Every variable lives in memory at a numbered location: its **address**. A **pointer** is a variable whose value *is* an address.

GUESS FIRST

`int number = 25;` and `int *ptr = &number;`. If you print `ptr`, do you see 25? If you print `*ptr`?



Exp 1's variables. The address 0x61FF08 is the manual's example; yours will differ on every run.

Symbol	Read it as	In Exp 1
`int *ptr;`	"ptr is a pointer to an int"	declares the pointer
`&number`	"address of number"	`ptr = &number;`
`*ptr`	"value at the address in ptr"	prints 25

`*` MEANS TWO DIFFERENT THINGS

In a **declaration** (`int *ptr;`), `*` means "this is a pointer". In an **expression** (`cout << *ptr;`), `*` means "go to that address" (dereference).

2.2 Exp 1 program (manual's code was correct)

E01_POINTERS.CPP

```cpp
#include <iostream>
using namespace std;

int main()
{
    int number = 25;
    int *ptr;              // ptr can hold the address of an int

    ptr = &number;         // & = "address of"

    cout << "Value of number = " << number << endl;
    cout << "Address of number = " << &number << endl;
    cout << "Pointer stores = " << ptr << endl;
    cout << "Value using pointer = " << *ptr << endl;   // * = "value at"

    return 0;
}
```

SCREEN (REAL RUN)

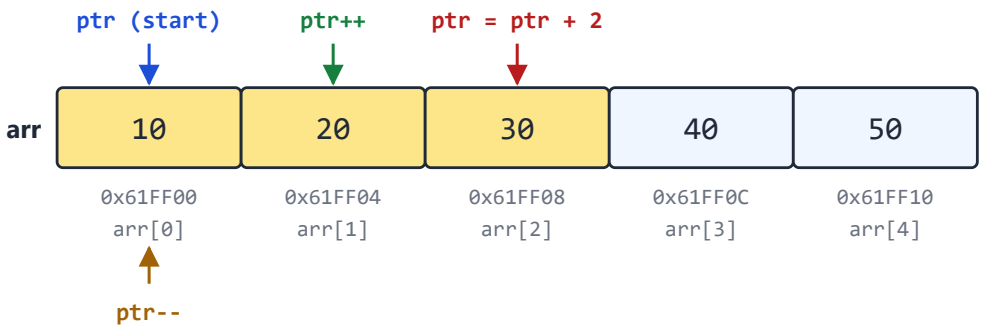
```
Value of number = 25
Address of number = 0xbb8cdff8c4
Pointer stores = 0xbb8cdff8c4
Value using pointer = 25
```

Lines 2 and 3 of the output always match each other; the actual number changes on every run and every PC. The manual says so too ("memory address varies").

2.3 Exp 2: pointer arithmetic

> **GUESS FIRST**
> `ptr` points at `arr[0]`, which is at address ...00. After `ptr++`, is the address ...01 or ...04? Why?

`ptr + 1` doesn't add 1 byte. It moves to the **next int**, which is `sizeof(int)` = 4 bytes further. That only works because an array's elements sit **side by side** in memory.



+1 on an int pointer = +4 bytes (sizeof(int)), i.e. the NEXT element

The manual's Figure 1, redrawn: each move is one whole element.

E02_POINTER_ARITHMETIC.CPP

```cpp
#include <iostream>
using namespace std;

int main()
{
    int arr[5] = {10, 20, 30, 40, 50};
    int *ptr = arr;                  // array name = address of arr[0]

    cout << "Initial Address : " << ptr << endl;
    cout << "Value : " << *ptr << endl;

    ptr++;                           // moves 1 int forward (4 bytes)
    cout << "\nAfter Increment" << endl;
    cout << "Address : " << ptr << endl;
    cout << "Value : " << *ptr << endl;

    ptr--;                           // moves 1 int back
    cout << "\nAfter Decrement" << endl;
    cout << "Address : " << ptr << endl;
    cout << "Value : " << *ptr << endl;

    ptr = ptr + 2;                   // moves 2 ints forward (8 bytes)
    cout << "\nAfter Adding 2" << endl;
    cout << "Address : " << ptr << endl;
    cout << "Value : " << *ptr << endl;

    return 0;
}
```

SCREEN (REAL RUN)

```
Initial Address : 0x45c67ffa90
Value : 10

After Increment
Address : 0x45c67ffa94
Value : 20

After Decrement
Address : 0x45c67ffa90
Value : 10

After Adding 2
Address : 0x45c67ffa98
Value : 30
```

The real run shows exactly the pattern the manual predicts: ...700 → ...704 → ...700 → ...708, values 10 → 20 → 10 → 30.

TWO FACTS THE VIVA LOVES

- `int *ptr = arr;` works because **an array's name is the address of its first element**: `arr == &arr[0]`.
- `*(ptr + i)` is the same as `arr[i]`. Square brackets are pointer arithmetic in disguise.

WRITE IN YOUR RECORD

Aim (Exp 1): To store the address of a variable in a pointer and access the variable through the pointer.

Algorithm: declare `int number = 25` → declare `int *ptr` → `ptr = &number` → print number, `&number`, `ptr`, `*ptr`.

Aim (Exp 2): To perform increment, decrement and addition on a pointer to an integer array.

Algorithm: `ptr = arr` → print → `ptr++` → print → `ptr--` → print → `ptr = ptr + 2` → print.

Complexity: time O(1), space O(1) for both.

Result (Exp 1): The pointer stored the address of `number` (same value as `&number`), and `*ptr` gave 25.

Result (Exp 2): Each increment moved the pointer by 4 bytes (one int): values 10, 20, 10, 30 at addresses base, +4, base, +8.

2.4 Viva questions

Question	Answer
What is a pointer?	A variable that stores the memory address of another variable.
What does `&` do? `*`?	`&x` gives x's address; `*p` gives the value stored at the address in p.
Why does `ptr++` add 4, not 1?	Pointer arithmetic moves in units of the pointed-to type; an int is 4 bytes here.
What would `ptr++` add for a `char *`? A `double *`?	1 byte; 8 bytes.
What is a NULL pointer?	A pointer that points nowhere (`ptr = NULL;`). Linked lists use it to mark the end.
What happens if you dereference an uninitialised pointer?	Undefined behaviour: it points to a random address, and the program may crash.
Where are pointers used?	Dynamic memory (`new`), linked lists, trees, graphs, passing by reference, arrays.

CHECK 2.1

`int a[4] = {5, 6, 7, 8}; int *p = a + 3;`. What is `*p`? What is `*(p - 2)`?

CHECK 2.2

Without looking, write the four lines of Exp 1 that declare `number`, declare `ptr`, assign it, and print the value through the pointer.

Answers

Check 2.1: `*p` = a[3] = **8**; `*(p - 2)` = a[1] = **6**.

Check 2.2: `int number = 25;` · `int *ptr;` · `ptr = &number;` · `cout << *ptr;`. If you wrote `ptr = number` (no `&`), the compiler rejects it: you can't store an int in a pointer.

3 · Exp 3 — Insert into an array (beginning, any position, end)

The first "data structure" experiment. The skill here, shifting elements in a loop, comes back in Exp 4 and inside insertion sort (Exp 7).

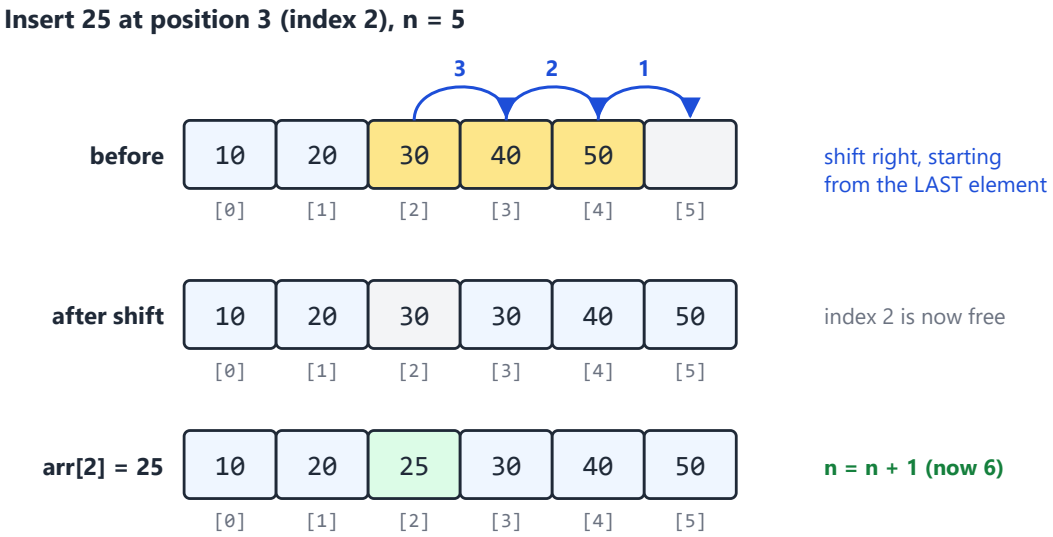
Array = fixed boxes → **Make a gap by shifting right** → Drop the value in → n = n + 1

3.1 The idea

An array's elements sit side by side and can't move by themselves. To put a value at index 2 you must first **make a gap** there by shifting everything from index 2 onward one place to the right.

GUESS FIRST

To shift `10 20 30 40 50` right from index 2, should the loop copy **30 first** or **50 first**? Try both on paper and see what goes wrong.



Insert 25 at position 3. The arrows are numbered in the order the loop does them.

If you copy 30 first, `arr[3] = arr[2]` overwrites 40 before 40 has moved: you get `10 20 30 30 30 30`. So the loop must run **from the end down**: `for (i = n; i >= pos; i--) arr[i] = arr[i-1];`.

POSITION VS INDEX

The program asks for a **position** counted from 1 (what humans say), but C++ indexes from 0. Position 3 = `arr[2]`, hence `arr[pos - 1] = value;`. Valid positions for insertion are **1 to n+1** (n+1 = "after the last one").

3.2 The program (corrected)

The manual's logic was right. This version adds the two checks it was missing: free space (`n < 100`) and a valid position.

E03_ARRAY_INSERT.CPP

```cpp
#include <iostream>
using namespace std;

int main()
{
    int arr[100], n, value, pos, choice;

    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter array elements:\n";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    if (n >= 100)                           // FIX: no room left for one more
    {
        cout << "Array is full. Insertion not possible.";
        return 0;
    }

    cout << "\n1. Insert at Beginning";
    cout << "\n2. Insert at Any Position";
    cout << "\n3. Insert at End";
    cout << "\nEnter your choice: ";
    cin >> choice;

    if (choice == 1)
    {
        cout << "Enter value: ";
        cin >> value;

        for (int i = n; i > 0; i--)         // shift right, last element first
            arr[i] = arr[i - 1];

        arr[0] = value;
        n++;
    }
    else if (choice == 2)
    {
        cout << "Enter Position: ";
        cin >> pos;

        if (pos < 1 || pos > n + 1)         // FIX: positions 1..n+1 are valid
        {
            cout << "Invalid position.";
            return 0;
        }

        cout << "Enter Value: ";
        cin >> value;

        for (int i = n; i >= pos; i--)      // shift right from the end down to pos
            arr[i] = arr[i - 1];

        arr[pos - 1] = value;               // position 3 = index 2
        n++;
    }
    else if (choice == 3)
    {
```

```
        cout << "Enter Value: ";
        cin >> value;

        arr[n] = value;                    // first free slot is index n
        n++;
    }
    else
    {
        cout << "Invalid choice.";
        return 0;
    }

    cout << "\nUpdated Array:\n";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";

    return 0;
}
```

The manual's three sample inputs, run for real:

SCREEN (REAL RUN)

```
Enter number of elements: 5
Enter array elements:
10 20 30 40 50

1. Insert at Beginning
2. Insert at Any Position
3. Insert at End
Enter your choice: 1
Enter value: 5

Updated Array:
5 10 20 30 40 50
```

SCREEN (REAL RUN)

```
Enter number of elements: 5
Enter array elements:
10 20 30 40 50

1. Insert at Beginning
2. Insert at Any Position
3. Insert at End
Enter your choice: 2
Enter Position: 3
Enter Value: 25

Updated Array:
10 20 25 30 40 50
```

SCREEN (REAL RUN)

```
Enter number of elements: 5
Enter array elements:
10 20 30 40 50

1. Insert at Beginning
2. Insert at Any Position
3. Insert at End
```

```
Enter your choice: 3
Enter Value: 60

Updated Array:
10 20 30 40 50 60
```

The added checks working (position 7 in a 3-element array):

```
SCREEN (REAL RUN)
Enter number of elements: 3
Enter array elements:
1 2 3

1. Insert at Beginning
2. Insert at Any Position
3. Insert at End
Enter your choice: 2
Enter Position: 7
Invalid position.
```

Case	Loop	Why
Beginning	`for (i = n; i > 0; i--) arr[i] = arr[i-1];` then `arr[0] = value;`	every element moves
Position	`for (i = n; i >= pos; i--) arr[i] = arr[i-1];` then `arr[pos-1] = value;`	elements from index pos−1 onward move
End	`arr[n] = value;`	first free slot is index n; nothing moves

All three end with `n++`: the array now holds one more element.

WRITE IN YOUR RECORD

Aim: To insert an element into an array at the beginning, at a given position, and at the end.

Algorithm: read n and the elements → read choice → *beginning:* shift all right (from the end), `arr[0] = value` → *position:* check 1 ≤ pos ≤ n+1, shift right from the end down to index pos−1, `arr[pos-1] = value` → *end:* `arr[n] = value` → `n++` → print.

Complexity: beginning O(n), position O(n) (worst case), end O(1). Space O(1) extra.

Result: 10 20 30 40 50 became 5 10 20 30 40 50 (beginning), 10 20 25 30 40 50 (25 at position 3), 10 20 30 40 50 60 (end).

MISTAKES THAT COST MARKS

- Forgetting `n++`: the new last element never prints.
- Shifting left-to-right: duplicates the value (see the guess above).
- `arr[pos] = value` instead of `arr[pos-1]`: everything lands one place too far right.

3.3 Viva questions

Question	Answer
Why is insertion at the beginning O(n)?	All n elements must shift one place right.
Why is insertion at the end O(1)?	No shifting: write at index n (if there is space).
What is the drawback of arrays for insertion?	Fixed size, and shifting costs O(n). A linked list inserts by changing pointers.
Why `int arr[100]` and not `int arr[n]`?	Array sizes must be constants in standard C++; 100 is the maximum capacity, n is how many are in use.
What if n is already 100?	Overflow: there's no room. The program must refuse (our first check).

3.4 If the examiner changes the question

The manual's ASSIGNMENT list. Each function is tested (10 checks, all pass) and is written to drop into the program above. Each returns the new n.

Insert at a position (reused by the next two):

```
int insertAt(int arr[], int n, int pos, int value)   // pos is 1-based
{
    if (n >= CAP || pos < 1 || pos > n + 1)
        return n;                                    // no space / bad position
    for (int i = n; i >= pos; i--)
        arr[i] = arr[i - 1];
    arr[pos - 1] = value;
    return n + 1;
}
```

Insert multiple elements:

```
// Insert k values (vals[0..k-1]) at the end, stopping if the array fills up
int insertMultiple(int arr[], int n, int vals[], int k)
{
    for (int i = 0; i < k; i++)
    {
        if (n >= CAP)
        {
            cout << "Array full, " << k - i << " value(s) not inserted" << endl;
            break;
        }
        arr[n] = vals[i];
        n++;
    }
    return n;
}
```

Insert into a sorted array, keeping it sorted. This is the core move of insertion sort (Exp 7):

```
// arr is sorted ascending; keep it sorted
int insertSorted(int arr[], int n, int value)
{
    if (n >= CAP)
        return n;
    int i = n - 1;
    while (i >= 0 && arr[i] > value)     // shift bigger ones right (like insertion sort)
    {
        arr[i + 1] = arr[i];
        i--;
    }
    arr[i + 1] = value;
    return n + 1;
}
```

Insert only if space is available:

```
int insertIfSpace(int arr[], int n, int value)
{
    if (n >= CAP)
    {
        cout << "No space available. Insertion not possible." << endl;
        return n;
    }
    arr[n] = value;                      // space exists: put it at the end
    return n + 1;
}
```

Insert before / after a given element: search for the element, then reuse `insertAt`.

```
// Insert value just before the first occurrence of target
int insertBefore(int arr[], int n, int target, int value)
{
    for (int i = 0; i < n; i++)
        if (arr[i] == target)
            return insertAt(arr, n, i + 1, value);   // index i = position i+1
    cout << target << " not found" << endl;
    return n;
}
```

```
int insertAfter(int arr[], int n, int target, int value)
{
    for (int i = 0; i < n; i++)
        if (arr[i] == target)
            return insertAt(arr, n, i + 2, value);   // one place further right
    cout << target << " not found" << endl;
    return n;
}
```

In `main` you call these like `n = insertSorted(arr, n, value);`.

> **CHECK 3.1**
> `arr = 4 8 15`, n = 3. Trace the "any position" branch for pos = 2, value = 6: which assignments run, in order, and what is the final array?

CHECK 3.2

Is `pos = 4` valid for this n = 3 array? Is `pos = 5` ?

Answers

Check 3.1: i = 3: `arr[3] = arr[2]` (15); i = 2: `arr[2] = arr[1]` (8); the loop stops (i = 1 < pos). Then `arr[1] = 6`, n = 4 → **4 6 8 15**.

Check 3.2: pos = 4 = n+1 is valid (insert at the end). pos = 5 is invalid: it would leave a gap at index 3.

4 · Exp 4 — Delete from an array (beginning, any position, end)

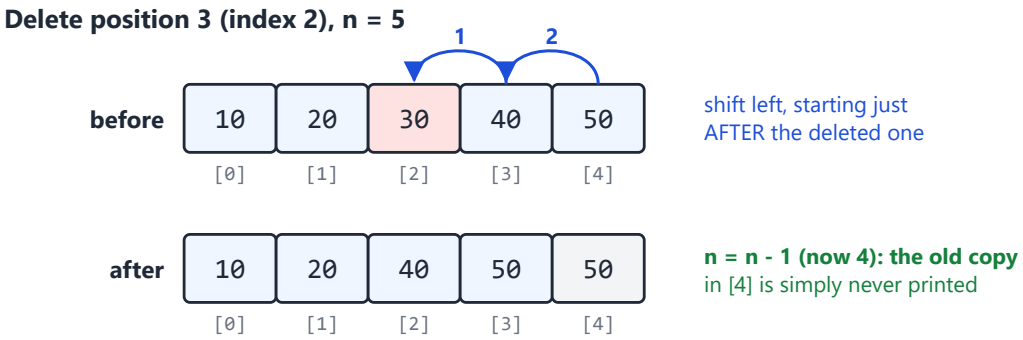
The mirror image of Exp 3: close the gap by shifting **left**, then n = n − 1.

Pick the element → **Shift the rest left** → n = n − 1 → print 0..n−1

4.1 The idea

GUESS FIRST

In Exp 3 the shift loop had to run from the **end** backwards. For deletion (shift left), which end must the loop start from, and why?



Delete position 3: shift from just after the gap, moving forwards.

When shifting **left**, start right after the deleted element and move **forward**: `arr[i] = arr[i+1]` for i = pos−1 … n−2. Going the other way would copy 50 over 40 before 40 has moved. **Rule of thumb: always copy the element that is about to be overwritten first.**

The last element is never erased; after `n--` the program simply never prints index n. That's why delete-at-end is just `n--`: O(1), no shifting.

4.2 The manual's bug

The PROGRAM listing (manual page 9) asks **"Enter value:"** in all three branches and reads a number it never uses. Deleting needs only a *position*. With the manual's own Sample Input 1 (`5`, `10 20 30 40 50`, `1`) that listing stops at "Enter value:" and waits for input the sample never gives. The IMPLEMENTATION screenshot (page 11) has no such line. This pack uses the screenshot's logic, plus a position check, and prints which element was deleted.

4.3 The program (corrected)

E04_ARRAY_DELETE.CPP

```
#include <iostream>
using namespace std;

int main()
```

```
{
    int arr[100], n, pos, choice;           // FIX: no "value" - deletion needs no value

    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter array elements:\n";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    if (n <= 0)                             // FIX: nothing to delete
    {
        cout << "Array is empty. Deletion not possible.";
        return 0;
    }

    cout << "\n1. Delete at Beginning";
    cout << "\n2. Delete at Any Position";
    cout << "\n3. Delete at End";
    cout << "\nEnter your choice: ";
    cin >> choice;

    if (choice == 1)
    {
        cout << "Deleted element: " << arr[0];
        for (int i = 0; i < n - 1; i++)     // shift left, first element first
            arr[i] = arr[i + 1];
        n--;
    }
    else if (choice == 2)
    {
        cout << "Enter Position: ";
        cin >> pos;

        if (pos < 1 || pos > n)             // FIX: positions 1..n are valid
        {
            cout << "Invalid position.";
            return 0;
        }

        cout << "Deleted element: " << arr[pos - 1];
        for (int i = pos - 1; i < n - 1; i++)
            arr[i] = arr[i + 1];
        n--;
    }
    else if (choice == 3)
    {
        cout << "Deleted element: " << arr[n - 1];
        n--;                                // just forget the last one
    }
    else
    {
        cout << "Invalid choice.";
        return 0;
    }

    cout << "\nUpdated Array:\n";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
```

```
    return 0;
}
```

The manual's three sample inputs:

SCREEN (REAL RUN)

```
Enter number of elements: 5
Enter array elements:
10 20 30 40 50

1. Delete at Beginning
2. Delete at Any Position
3. Delete at End
Enter your choice: 1
Deleted element: 10
Updated Array:
20 30 40 50
```

SCREEN (REAL RUN)

```
Enter number of elements: 5
Enter array elements:
10 20 30 40 50

1. Delete at Beginning
2. Delete at Any Position
3. Delete at End
Enter your choice: 2
Enter Position: 3
Deleted element: 30
Updated Array:
10 20 40 50
```

SCREEN (REAL RUN)

```
Enter number of elements: 5
Enter array elements:
10 20 30 40 50

1. Delete at Beginning
2. Delete at Any Position
3. Delete at End
Enter your choice: 3
Deleted element: 50
Updated Array:
10 20 30 40
```

A bad position is refused:

SCREEN (REAL RUN)

```
Enter number of elements: 5
Enter array elements:
10 20 30 40 50

1. Delete at Beginning
2. Delete at Any Position
3. Delete at End
Enter your choice: 2
```

```
Enter Position: 6
Invalid position.
```

Case	Loop	Then
Beginning	`for (i = 0; i < n-1; i++) arr[i] = arr[i+1];`	`n--`
Position	`for (i = pos-1; i < n-1; i++) arr[i] = arr[i+1];`	`n--`
End	(none)	`n--`

Valid deletion positions are **1 to n** (not n+1: there's nothing there to delete). With n = 0 there's nothing to delete at all: **underflow**.

> **WRITE IN YOUR RECORD**
>
> **Aim:** To delete an element from an array at the beginning, at a given position, and at the end.
>
> **Algorithm:** read n and the elements → read choice → *beginning:* shift elements 1…n−1 one place left → *position:* check 1 ≤ pos ≤ n, shift elements after pos−1 one place left → *end:* nothing to shift → `n--` → print.
>
> **Complexity:** beginning O(n), position O(n) (worst case), end O(1). Space O(1).
>
> **Result:** 10 20 30 40 50 became 20 30 40 50 (beginning), 10 20 40 50 (position 3, deleted 30), 10 20 30 40 (end).

4.4 Viva questions

Question	Answer
Why does deletion at the end need no shifting?	Reducing n hides the last element; no gap is created.
Is the deleted value really erased from memory?	No. It stays in the slot until overwritten; the program just stops counting it.
What is underflow here?	Trying to delete from an empty array (n = 0).
Insertion shifts from the end, deletion from the front. Why?	So no element is overwritten before it has been copied.
Time complexity of deleting a value you first have to search for?	O(n): the search is O(n) and the shift is O(n), so O(n) overall.

4.5 If the examiner changes the question

Beginning, position and end are the experiment itself. The other two items:

Delete a given value (search, then shift):

```c
// Search for value; if found, delete it (first occurrence) and shift left
int deleteByValue(int arr[], int n, int value)
{
    int pos = -1;
    for (int i = 0; i < n; i++)          // linear search
```

```
        if (arr[i] == value)
        {
            pos = i;
            break;
        }

    if (pos == -1)
    {
        cout << value << " not found in array" << endl;
        return n;
    }
    for (int i = pos; i < n - 1; i++)
        arr[i] = arr[i + 1];
    return n - 1;
}
```

Menu-driven insertion and deletion on one array. A full program combining Exp 3 and Exp 4, with a repeating menu (the `do … while` pattern from Exp 10):

E04B_ARRAY_MENU.CPP

```
// Exp 4 assignment: menu-driven insertion AND deletion on one array
#include <iostream>
using namespace std;

int arr[100], n = 0;

void display()
{
    if (n == 0)
        cout << "Array is empty." << endl;
    else
    {
        cout << "Array: ";
        for (int i = 0; i < n; i++)
            cout << arr[i] << " ";
        cout << endl;
    }
}

void insertAt(int pos, int value)            // pos is 1-based
{
    if (n >= 100)
        cout << "Array is full." << endl;
    else if (pos < 1 || pos > n + 1)
        cout << "Invalid position." << endl;
    else
    {
        for (int i = n; i >= pos; i--)
            arr[i] = arr[i - 1];
        arr[pos - 1] = value;
        n++;
    }
}

void deleteAt(int pos)
{
    if (n == 0)
        cout << "Array is empty." << endl;
    else if (pos < 1 || pos > n)
```

```
        cout << "Invalid position." << endl;
    else
    {
        cout << "Deleted element: " << arr[pos - 1] << endl;
        for (int i = pos - 1; i < n - 1; i++)
            arr[i] = arr[i + 1];
        n--;
    }
}

int main()
{
    int choice, pos, value;
    do
    {
        cout << "\n1. Insert  2. Delete  3. Display  4. Exit" << endl;
        cout << "Enter your choice: ";
        cin >> choice;

        if (choice == 1)
        {
            cout << "Enter position and value: ";
            cin >> pos >> value;
            insertAt(pos, value);
        }
        else if (choice == 2)
        {
            cout << "Enter position: ";
            cin >> pos;
            deleteAt(pos);
        }
        else if (choice == 3)
            display();
        else if (choice != 4)
            cout << "Invalid choice!" << endl;
    } while (choice != 4);

    return 0;
}
```

SCREEN (REAL RUN; MENU SHOWN ONCE, IT REPRINTS BEFORE EACH CHOICE)

```
1. Insert  2. Delete  3. Display  4. Exit
Enter your choice: 1
Enter position and value: 1 10
Enter your choice: 1
Enter position and value: 2 30
Enter your choice: 1
Enter position and value: 2 20
Enter your choice: 3
Array: 10 20 30
Enter your choice: 2
Enter position: 1
Deleted element: 10
Enter your choice: 3
Array: 20 30
Enter your choice: 2
Enter position: 5
Invalid position.
Enter your choice: 4
```

CHECK 4.1

`arr` = `7 3 9 1`, n = 4. Delete position 2. Write the assignments the loop performs, the final n, and the printed array.

CHECK 4.2

Someone writes the position loop as `for (i = pos-1; i < n; i++) arr[i] = arr[i+1];`. What goes wrong?

Answers

Check 4.1: i = 1: `arr[1] = arr[2]` (9); i = 2: `arr[2] = arr[3]` (1); stop at i = 3. n = 3 → **7 9 1**.

Check 4.2: On the last pass (i = n−1) it reads `arr[n]`, one past the data: junk (or past the end of the array if n = 100). The result still prints correctly, because n−1 elements are printed, but it reads memory it shouldn't. The correct bound is `i < n-1`.

5 · Exp 5 + 6 — Matrix multiplication and transpose (modular programming)

Two experiments, one pattern: three functions (input, operate, display) plus a main() that calls them. New C++ here: 2-D arrays and passing them to functions.

2-D array → Pass to a function → inputMatrix → multiply / transpose → displayMatrix

5.1 New C++: 2-D arrays

`int A[10][10];` is a grid of 10 rows × 10 columns; `A[i][j]` is row i, column j (both from 0). Like `arr[100]`, the size is the **maximum**: the user's 2 × 3 matrix uses only the top-left corner.

Walking a grid takes **two nested loops**: the outer loop picks the row, the inner loop walks along it.

```
for (int i = 0; i < rows; i++)
    for (int j = 0; j < cols; j++)
        cin >> A[i][j];
```

WHY THE PARAMETER IS `INT MATRIX[][10]`

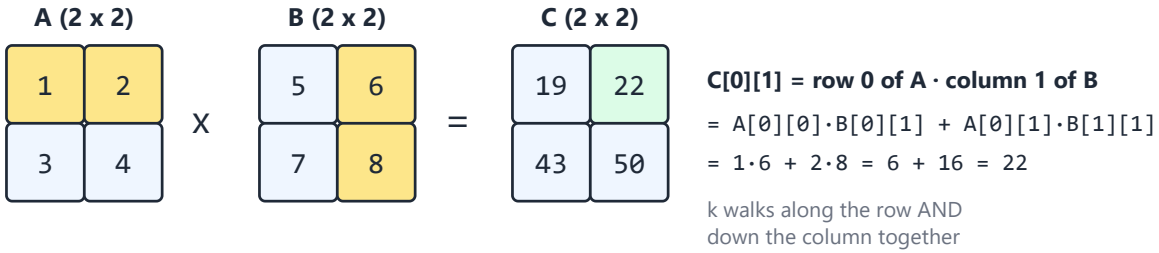
When passing a 2-D array, the **number of columns must be written** and must match the declaration (`[10]`). Only the first size may be left empty. `int matrix[][]` doesn't compile.

Modular programming means splitting the program into functions that each do one job: `inputMatrix()`, `multiplyMatrix()` / `transposeMatrix()`, `displayMatrix()`, with `main()` only reading sizes and calling them. The manual asks for exactly this structure, so use these exact function names.

5.2 Exp 5: multiplication

GUESS FIRST

A is 2 × **3** and B is **3** × 2. Can you multiply A × B? What order is the answer? Can you multiply B × A?



Rule: (m x n) times (n x p) gives (m x p). The two inner numbers must match: c1 == r2.

Each answer cell is one row of A "dotted" with one column of B.

$$C[i][j] = A[i][0]\cdot B[0][j] + A[i][1]\cdot B[1][j] + \ldots + A[i][c1-1]\cdot B[c1-1][j]$$

That sum is the **third loop** (k). Three nested loops: i over rows of A, j over columns of B, k along the shared dimension. Hence **$O(n^3)$** for n × n matrices.

E05_MATRIX_MULTIPLY.CPP

```cpp
#include <iostream>
using namespace std;

// Function to input matrix elements
void inputMatrix(int matrix[][10], int rows, int cols)
{
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            cin >> matrix[i][j];
}

// Function to multiply two matrices: C = A x B
void multiplyMatrix(int A[][10], int B[][10], int C[][10], int r1, int c1, int c2)
{
    for (int i = 0; i < r1; i++)              // each row of A
    {
        for (int j = 0; j < c2; j++)          // each column of B
        {
            C[i][j] = 0;
            for (int k = 0; k < c1; k++)      // walk along row i and column j
                C[i][j] = C[i][j] + A[i][k] * B[k][j];
        }
    }
}

// Function to display matrix
void displayMatrix(int matrix[][10], int rows, int cols)
{
    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
            cout << matrix[i][j] << " ";
        cout << endl;
    }
}

int main()
{
    int A[10][10], B[10][10], C[10][10];
    int r1, c1, r2, c2;

    cout << "Enter rows and columns of first matrix: ";
    cin >> r1 >> c1;

    cout << "Enter rows and columns of second matrix: ";
    cin >> r2 >> c2;

    // FIX: arrays are 10 x 10, so larger sizes would overflow
    if (r1 < 1 || c1 < 1 || r2 < 1 || c2 < 1 || r1 > 10 || c1 > 10 || r2 > 10 || c2 > 10)
    {
        cout << "Sizes must be between 1 and 10.";
        return 0;
    }

    // Check multiplication condition
    if (c1 != r2)
```

```
    {
        cout << "Matrix multiplication not possible.";
        return 0;
    }

    cout << "Enter elements of first matrix:\n";
    inputMatrix(A, r1, c1);

    cout << "Enter elements of second matrix:\n";
    inputMatrix(B, r2, c2);

    multiplyMatrix(A, B, C, r1, c1, c2);

    cout << "\nResultant Matrix:\n";
    displayMatrix(C, r1, c2);

    return 0;
}
```

The manual's sample (2 × 2), and a non-square case worked by hand (row 1 of the answer: 1·7 + 2·9 + 3·11 = 58):

SCREEN (REAL RUN)

```
Enter rows and columns of first matrix: 2 2
Enter rows and columns of second matrix: 2 2
Enter elements of first matrix:
1 2
3 4
Enter elements of second matrix:
5 6
7 8

Resultant Matrix:
19 22
43 50
```

SCREEN (REAL RUN)

```
Enter rows and columns of first matrix: 2 3
Enter rows and columns of second matrix: 3 2
Enter elements of first matrix:
1 2 3
4 5 6
Enter elements of second matrix:
7 8
9 10
11 12

Resultant Matrix:
58 64
139 154
```

SCREEN (REAL RUN)

```
Enter rows and columns of first matrix: 2 3
Enter rows and columns of second matrix: 2 3
Matrix multiplication not possible.
```

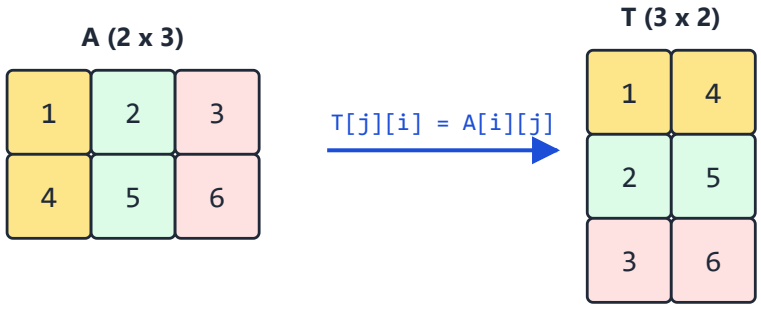
THE LINE PEOPLE FORGET

`C[i][j] = 0;` before the k-loop. Without it, C starts with garbage and the sum is wrong. It must sit **inside** the j-loop, so each cell starts from 0.

5.3 Exp 6: transpose

GUESS FIRST

A is 2 × 3. What is the order of its transpose, and where does A[0][2] go?



Column j of A becomes row j of T (each colour = one column of A → one row of T)

Rows become columns: T[j][i] = A[i][j].

E06_MATRIX_TRANSPOSE.CPP

```cpp
#include <iostream>
using namespace std;

// Function to input matrix elements
void inputMatrix(int matrix[][10], int rows, int cols)
{
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            cin >> matrix[i][j];
}

// Function to find transpose of matrix
void transposeMatrix(int A[][10], int T[][10], int rows, int cols)
{
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            T[j][i] = A[i][j];              // row i becomes column i
}

// Function to display matrix
void displayMatrix(int matrix[][10], int rows, int cols)
{
    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
            cout << matrix[i][j] << " ";
        cout << endl;
    }
}
```

```
int main()
{
    int A[10][10], T[10][10];
    int rows, cols;

    cout << "Enter number of rows and columns of matrix: ";
    cin >> rows >> cols;

    if (rows < 1 || cols < 1 || rows > 10 || cols > 10)   // FIX: array is 10 x 10
    {
        cout << "Sizes must be between 1 and 10.";
        return 0;
    }

    cout << "Enter elements of matrix:\n";
    inputMatrix(A, rows, cols);

    transposeMatrix(A, T, rows, cols);

    cout << "\nOriginal Matrix:\n";
    displayMatrix(A, rows, cols);

    cout << "\nTranspose Matrix:\n";
    displayMatrix(T, cols, rows);            // rows and columns swap

    return 0;
}
```

SCREEN (REAL RUN)

```
Enter number of rows and columns of matrix: 2 3
Enter elements of matrix:
1 2 3
4 5 6

Original Matrix:
1 2 3
4 5 6

Transpose Matrix:
1 4
2 5
3 6
```

Note the call `displayMatrix(T, cols, rows)`: the transpose has the sizes **swapped**. Passing `(T, rows, cols)` prints the wrong shape.

WRITE IN YOUR RECORD — EXP 5

Aim: To multiply two matrices using separate functions for input, multiplication and display.

Algorithm: read r1, c1, r2, c2 → if c1 ≠ r2, print "not possible" and stop → `inputMatrix(A)`, `inputMatrix(B)` → `multiplyMatrix`: for each i, j set C[i][j] = 0, then for k = 0…c1−1 add A[i][k]·B[k][j] → `displayMatrix(C, r1, c2)`.

Complexity: time O(r1·c1·c2), i.e. O(n^3) for square matrices; space O(r1·c2) for C.

Result: [[1,2],[3,4]] × [[5,6],[7,8]] = [[19,22],[43,50]].

WRITE IN YOUR RECORD — EXP 6

Aim: To find the transpose of a matrix using separate functions for input, transpose and display.

Algorithm: read rows, cols → `inputMatrix(A)` → `transposeMatrix`: for each i, j set T[j][i] = A[i][j] → display A (rows × cols) and T (cols × rows).

Complexity: time O(m·n); space O(m·n) for T.

Result: [[1,2,3],[4,5,6]] (2 × 3) → [[1,4],[2,5],[3,6]] (3 × 2).

5.4 Viva questions

Question	Answer
Condition for A × B?	Columns of A = rows of B (c1 == r2). Result is r1 × c2.
Is A × B = B × A?	Generally no; matrix multiplication isn't commutative (B × A may not even be defined).
Why three loops?	Two pick the answer cell (i, j); the third (k) adds up the row·column products.
Why `int matrix[][10]`?	The compiler needs the column count to find `matrix[i][j]` in memory (row i starts i·10 elements in).
Advantage of modular programming?	Each function is small and testable; `inputMatrix` is reused for A and B; easier to debug and modify.
Can a square matrix be transposed in place?	Yes: swap A[i][j] with A[j][i] for j > i only (the upper triangle). A rectangular one can't, because its shape changes.
What is a symmetric matrix?	A square matrix equal to its transpose (A[i][j] = A[j][i]).

5.5 If the examiner changes the question

"Multiply two 3 × 3 matrices", "check whether they can be multiplied" and "transpose of a 3 × 3" are all the programs above (type 3 3 as the sizes). The new ones:

Add / subtract (same order needed: r1 == r2 and c1 == c2, checked in main):

```
// Both matrices must have the SAME order (rows x cols)
void addMatrix(int A[][10], int B[][10], int S[][10], int rows, int cols)
{
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            S[i][j] = A[i][j] + B[i][j];
}
```

```
void subtractMatrix(int A[][10], int B[][10], int D[][10], int rows, int cols)
{
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            D[i][j] = A[i][j] - B[i][j];
}
```

Symmetric or not:

```
// Symmetric = square AND A[i][j] == A[j][i] everywhere (A equals its transpose)
bool isSymmetric(int A[][10], int rows, int cols)
{
    if (rows != cols)
        return false;
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            if (A[i][j] != A[j][i])
                return false;
    return true;
}
```

In `main`: `if (isSymmetric(A, rows, cols)) cout << "Symmetric"; else cout << "Not symmetric";`

> **CHECK 5.1**
> A = [[2, 0], [1, 3]], B = [[1, 4], [5, 2]]. Compute C = A × B by hand, cell by cell.

> **CHECK 5.2**
> In `multiplyMatrix`, someone writes `for (int k = 0; k < c2; k++)`. For 2 × 3 times 3 × 2 matrices, what goes wrong?

Answers

Check 5.1: C[0][0] = 2·1 + 0·5 = 2; C[0][1] = 2·4 + 0·2 = 8; C[1][0] = 1·1 + 3·5 = 16; C[1][1] = 1·4 + 3·2 = 10 → **[[2, 8], [16, 10]]**.

Check 5.2: k runs only to 1 (c2 = 2) instead of 2, so the third product A[i][2]·B[2][j] is never added and every cell is wrong. k must run over the **shared** size, c1 (= r2).

6 · Exp 7 — Insertion sort (and its performance)

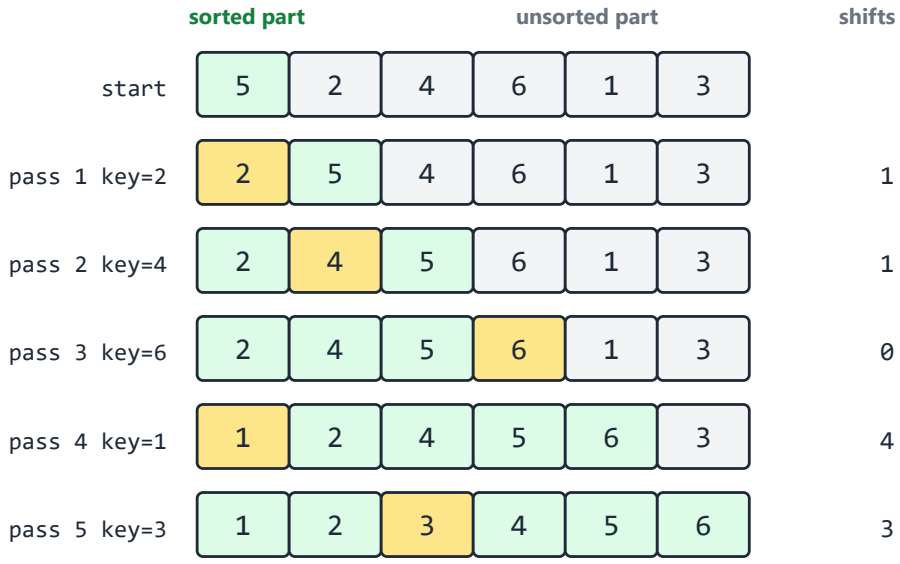
You already know the core move from Exp 3's "insert into a sorted array". Insertion sort just does it for every element in turn.

Sorted part / unsorted part → Pick key → **Shift bigger ones right** → Drop key in gap → Repeat

6.1 The idea

Like sorting playing cards in your hand: the left part is already sorted. Pick up the next card (the **key**), slide every bigger card one place right, and drop the key into the gap.

> **GUESS FIRST**
>
> `5 2 4 6 1 3`. How many single-place shifts does insertion sort need in total? (Count them pass by pass.) And how many for the already-sorted `1 2 3 4 5 6`?



yellow = where the key landed; total shifts = 9

The manual's example, pass by pass. Green = sorted part; yellow = where the key landed.

```
int key = arr[i];
int j = i - 1;
while (j >= 0 && arr[j] > key)   // bigger than key? move it right
{
    arr[j + 1] = arr[j];
    j--;
}
arr[j + 1] = key;                // j stopped one left of the gap
```

WHY `ARR[J + 1] = KEY` AND NOT `ARR[J]`

The loop stops when `arr[j]` is **not** bigger than the key (or j = −1). The gap is one to the right of that element, at j + 1.

Why `j >= 0` comes **first**: when the key is the smallest (key = 1 in pass 4), j reaches −1. `&&` stops at the first false, so `arr[-1]` is never read. Swap the order and the program reads outside the array.

6.2 The program (corrected)

The manual's sort was correct. The only fix: `int arr[n]` (a size read at run time) isn't standard C++, and strict compilers reject it:

G++ SAYS

```
m07_vla.cpp:25:9: error: ISO C++ forbids variable length array 'arr' [-Wvla]
   25 |     int arr[n];
      |         ^~~
```

So the array is `int arr[100]` with a range check on n.

E07_INSERTION_SORT.CPP

```cpp
#include <iostream>
using namespace std;

// Function to perform Insertion Sort
void insertionSort(int arr[], int n)
{
    for (int i = 1; i < n; i++)
    {
        int key = arr[i];                    // card picked up
        int j = i - 1;

        // Shift elements greater than key one position to the right
        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            j--;
        }

        arr[j + 1] = key;                    // drop the card in the gap
    }
}

// Function to display the array
void displayArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
}

int main()
{
    int n;
    int arr[100];                            // FIX: was int arr[n] (not standard C++)
```

```
    cout << "Enter number of elements: ";
    cin >> n;

    if (n < 1 || n > 100)
    {
        cout << "n must be between 1 and 100.";
        return 0;
    }

    cout << "Enter elements of the array:" << endl;
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    cout << "Original Array: ";
    displayArray(arr, n);

    insertionSort(arr, n);

    cout << "Sorted Array: ";
    displayArray(arr, n);

    return 0;
}
```

SCREEN (REAL RUN)

```
Enter number of elements: 6
Enter elements of the array:
5 2 4 6 1 3
Original Array: 5 2 4 6 1 3
Sorted Array: 1 2 3 4 5 6
```

Duplicates and negatives work too (the strict `>` keeps equal elements in their original order, which is why the sort is **stable**):

SCREEN (REAL RUN)

```
Enter number of elements: 7
Enter elements of the array:
3 -1 3 0 9 -1 2
Original Array: 3 -1 3 0 9 -1 2
Sorted Array: -1 -1 0 2 3 3 9
```

6.3 Performance analysis (the "analyse" half of the aim)

These counts come from the tested counting version (section 6.5):

Input (n = 6)	Comparisons	Shifts	Case
1 2 3 4 5 6 (sorted)	5 = n − 1	0	**best**: O(n)
5 2 4 6 1 3 (manual)	12	9	average: $O(n^2)$
6 5 4 3 2 1 (reversed)	15 = n(n−1)/2	15	**worst**: $O(n^2)$

- **Best case O(n):** each key is compared once with its left neighbour, and stops.
- **Worst case O(n^2):** key i is compared with and shifted past all i elements before it: 1 + 2 + … + (n−1) = n(n−1)/2.
- **Space O(1):** sorts inside the same array using only `key` and `j`. It's **in-place**.

WRITE IN YOUR RECORD

Aim: To sort an integer array in ascending order using insertion sort and analyse its performance.

Algorithm: for i = 1 to n−1: key = arr[i], j = i−1; while j ≥ 0 and arr[j] > key: arr[j+1] = arr[j], j−−; then arr[j+1] = key. Print the array.

Complexity: best O(n) (sorted input), average and worst O(n^2) (reverse-sorted input: n(n−1)/2 comparisons); space O(1).

Result: 5 2 4 6 1 3 → 1 2 3 4 5 6 in 5 passes (9 shifts). Insertion sort is in-place, stable, and fast for small or nearly sorted data.

6.4 Viva questions

Question	Answer
Why does the outer loop start at i = 1?	A single element (index 0) is already "sorted".
Best, average, worst case?	O(n), O(n^2), O(n^2).
What input gives the best case? The worst?	Already sorted; reverse sorted.
Is it stable? In-place?	Both yes: `>` (not `>=`) keeps equal keys in order; only O(1) extra memory.
When is insertion sort a good choice?	Small arrays, nearly-sorted data, or data arriving one item at a time ("online").
Why save arr[i] in key?	The first shift overwrites arr[i]; without the copy the value would be lost.
How many passes for n elements?	n − 1.

6.5 If the examiner changes the question

"Separate functions for input, sorting and display" is the program above (add an `inputArray` function if asked). The rest:

Descending order (one character changes):

```
// Descending: the ONLY change is  arr[j] > key  ->  arr[j] < key
void insertionSortDesc(int arr[], int n)
{
    for (int i = 1; i < n; i++)
    {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] < key)
        {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}
```

Count comparisons and shifts. Used for the table in 6.3, and covers "compare performance for sorted, reverse-sorted and random arrays": run it on all three and print the counts.

```
// Counts comparisons (each arr[j] > key test) and shifts (each arr[j+1] = arr[j])
void insertionSortCount(int arr[], int n, int &comparisons, int &shifts)
{
    comparisons = 0;
    shifts = 0;
    for (int i = 1; i < n; i++)
    {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0)
        {
            comparisons++;
            if (arr[j] > key)
            {
                arr[j + 1] = arr[j];
                shifts++;
                j--;
            }
            else
                break;
        }
        arr[j + 1] = key;
    }
}
```

Call: `int c, s; insertionSortCount(arr, n, c, s); cout << "Comparisons = " << c << ", Shifts = " << s;`

Floating-point numbers:

```c
// Floating-point version: change the type of the array and of key - nothing else
void insertionSortFloat(float arr[], int n)
{
    for (int i = 1; i < n; i++)
    {
        float key = arr[i];
        int j = i - 1;                      // j is an index: stays int
        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}
```

Position of an element after sorting:

```c
// After sorting, find where x ended up (1-based position, -1 if absent)
int findPosition(int arr[], int n, int x)
{
    for (int i = 0; i < n; i++)
        if (arr[i] == x)
            return i + 1;
    return -1;
}
```

CHECK 6.1

Sort `4 3 2 10 12 1 5 6` by hand, pass by pass. Write the array after each pass.

CHECK 6.2

Change one character in the while condition to sort in **descending** order. Which character, and what does it become?

Answers

Check 6.1: key 3 → `3 4 2 10 12 1 5 6`; key 2 → `2 3 4 10 12 1 5 6`; key 10 → no change; key 12 → no change; key 1 → `1 2 3 4 10 12 5 6`; key 5 → `1 2 3 4 5 10 12 6`; key 6 → `1 2 3 4 5 6 10 12`. Seven passes (n − 1).

Check 6.2: `arr[j] > key` → `arr[j] < key`: move **smaller** elements right, so bigger ones end up first.

7 · Exp 8 — Quick sort, and comparison with other sorts

The hardest program in the lab: recursion + partition. Learn partition first; once that's solid, quickSort itself is 6 lines.

Pick pivot (last) → **Partition: small left, big right** → Pivot is now final → Recurse on left part → Recurse on right part

7.1 The idea: divide and conquer

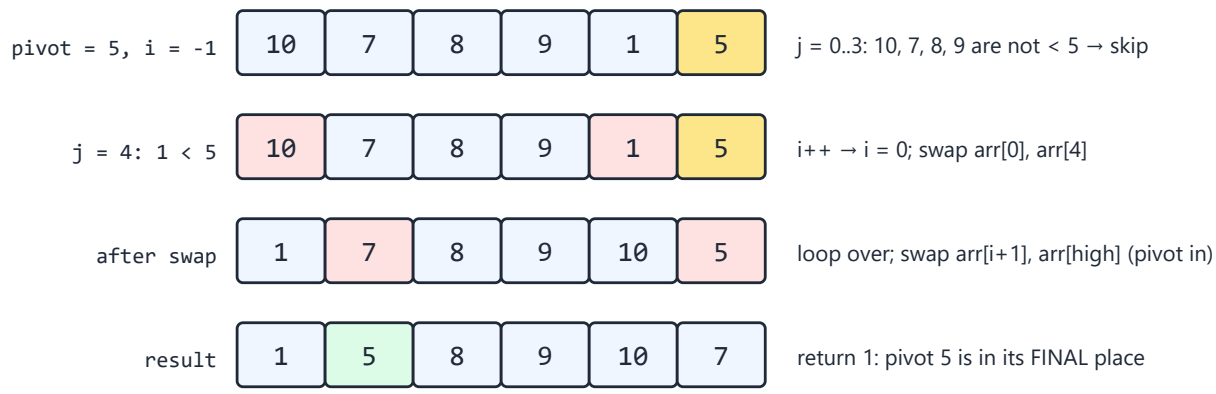
1. **Pick a pivot.** This program uses the **last** element.
2. **Partition:** rearrange so everything smaller than the pivot is on its left and everything else on its right. The pivot is then in its **final sorted position**.
3. **Recurse:** quick-sort the left part and the right part the same way. A part of 0 or 1 element is already sorted, and the recursion stops there.

7.2 Partition, step by step (learn this cold)

GUESS FIRST

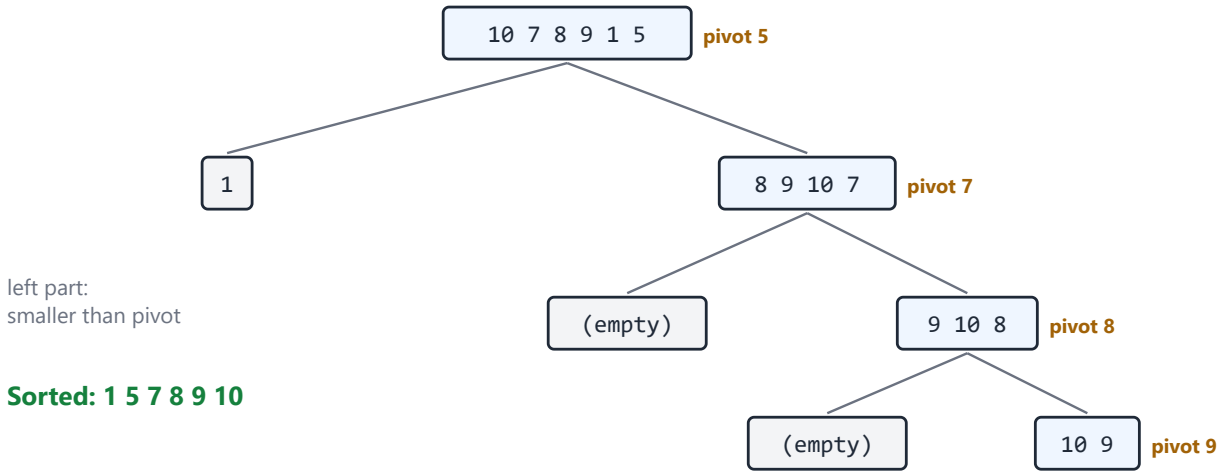
In `partitionArray`, the variable `i` starts at `low - 1`. What does `i` keep track of? (Hint: watch where small elements get swapped to.)

```
int pivot = arr[high];
int i = low - 1;                // last index of the "smaller than pivot" zone
for (int j = low; j < high; j++)
    if (arr[j] < pivot)         // small one found: grow the zone, swap it in
    {
        i++;
        swapElements(arr[i], arr[j]);
    }
swapElements(arr[i + 1], arr[high]);   // pivot goes right after the zone
return i + 1;                          // pivot's final index
```



yellow = pivot, red = the two cells being swapped, green = pivot placed

First partition of the manual's example 10 7 8 9 1 5. Same result as the manual's "Pass 1".



The whole recursion. Each box is one call to partition; the manual's Passes 1–4 are these four pivots.

WHAT `INT &A` DOES IN SWAPELEMENTS

`swapElements(arr[i], arr[j])` must change the array itself. `int &a` makes `a` **another name** for `arr[i]` (a reference, chapter 1.5). With plain `int a`, the function would swap two copies and the array would not change.

7.3 The program (corrected)

Same VLA fix as Exp 7 (`int arr[100]`). Partition and recursion are exactly the manual's.

E08_QUICK_SORT.CPP

```cpp
#include <iostream>
using namespace std;

// Function to swap two elements (& = work on the caller's variables)
void swapElements(int &a, int &b)
{
    int temp = a;
    a = b;
    b = temp;
}

// Function to partition the array around the last element
int partitionArray(int arr[], int low, int high)
{
    int pivot = arr[high];                  // last element as pivot
    int i = low - 1;                        // end of the "smaller than pivot" zone

    for (int j = low; j < high; j++)
    {
        if (arr[j] < pivot)
        {
            i++;
            swapElements(arr[i], arr[j]);
        }
    }
    swapElements(arr[i + 1], arr[high]);    // pivot goes just after the small zone
    return i + 1;                           // pivot's final index
}
```

```cpp
// Function to implement Quick Sort
void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int pi = partitionArray(arr, low, high);
        quickSort(arr, low, pi - 1);         // sort left part
        quickSort(arr, pi + 1, high);        // sort right part
    }
}

// Function to display the array
void displayArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
}

int main()
{
    int n;
    int arr[100];                            // FIX: was int arr[n] (not standard C++)

    cout << "Enter number of elements: ";
    cin >> n;

    if (n < 1 || n > 100)
    {
        cout << "n must be between 1 and 100.";
        return 0;
    }

    cout << "Enter elements of the array:" << endl;
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    cout << "Original Array: ";
    displayArray(arr, n);

    quickSort(arr, 0, n - 1);

    cout << "Sorted Array using Quick Sort: ";
    displayArray(arr, n);

    return 0;
}
```

SCREEN (REAL RUN)

```
Enter number of elements: 6
Enter elements of the array:
10 7 8 9 1 5
Original Array: 10 7 8 9 1 5
Sorted Array using Quick Sort: 1 5 7 8 9 10
```

With duplicates and negatives:

SCREEN (REAL RUN)

```
Enter number of elements: 8
Enter elements of the array:
4 4 1 9 0 4 -3 7
Original Array: 4 4 1 9 0 4 -3 7
Sorted Array using Quick Sort: -3 0 1 4 4 4 7 9
```

7.4 The missing half: comparing with other sorts

The aim says "compare its performance with other sorting techniques". The manual's algorithm (steps 11–12) and explanation table list `bubbleSort()`, `selectionSort()`, `insertionSort()`, `copyArray()` and `clock()`, but **none of them are in the printed code**. Here is the full program:

- Every sort gets its **own copy** of the same input (`copyArray`). Sorting the already-sorted output again wouldn't be a fair test.
- Timing **6 numbers** gives 0 ms for everything, so part 2 times 10 000 random numbers. `srand(1)` makes the "random" data the same on every run.
- `clock()` counts processor ticks; `(end - start) / CLOCKS_PER_SEC` converts to seconds, and × 1000 to milliseconds.

E08_SORT_COMPARE.CPP

```cpp
#include <iostream>
#include <cstdlib>     // rand, srand
#include <ctime>       // clock, CLOCKS_PER_SEC
using namespace std;

const int BIG = 10000;                      // size used for the timing test

void swapElements(int &a, int &b)
{
    int temp = a;
    a = b;
    b = temp;
}

int partitionArray(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++)
        if (arr[j] < pivot)
        {
            i++;
            swapElements(arr[i], arr[j]);
        }
    swapElements(arr[i + 1], arr[high]);
    return i + 1;
}

void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int pi = partitionArray(arr, low, high);
        quickSort(arr, low, pi - 1);
```

```
        quickSort(arr, pi + 1, high);
    }
}

// Bubble Sort: neighbours swap until the largest "bubbles" to the end
void bubbleSort(int arr[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        bool swapped = false;
        for (int j = 0; j < n - 1 - i; j++)
            if (arr[j] > arr[j + 1])
            {
                swapElements(arr[j], arr[j + 1]);
                swapped = true;
            }
        if (!swapped)                       // no swap in a pass = already sorted
            break;
    }
}

// Selection Sort: pick the smallest of the rest, put it at position i
void selectionSort(int arr[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        int minIndex = i;
        for (int j = i + 1; j < n; j++)
            if (arr[j] < arr[minIndex])
                minIndex = j;
        swapElements(arr[i], arr[minIndex]);
    }
}

void insertionSort(int arr[], int n)
{
    for (int i = 1; i < n; i++)
    {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

void copyArray(int source[], int dest[], int n)
{
    for (int i = 0; i < n; i++)
        dest[i] = source[i];
}

void displayArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
}
```

```
int original[BIG], work[BIG];               // global: too big for the stack

int main()
{
    int n;
    cout << "Enter number of elements: ";
    cin >> n;

    if (n < 1 || n > 100)
    {
        cout << "n must be between 1 and 100.";
        return 0;
    }

    cout << "Enter elements of the array:" << endl;
    for (int i = 0; i < n; i++)
        cin >> original[i];

    // Part 1: every algorithm sorts the SAME input (each gets its own copy)
    copyArray(original, work, n);  quickSort(work, 0, n - 1);
    cout << "Quick Sort     : ";  displayArray(work, n);
    copyArray(original, work, n);  bubbleSort(work, n);
    cout << "Bubble Sort    : ";  displayArray(work, n);
    copyArray(original, work, n);  selectionSort(work, n);
    cout << "Selection Sort : ";  displayArray(work, n);
    copyArray(original, work, n);  insertionSort(work, n);
    cout << "Insertion Sort : ";  displayArray(work, n);

    // Part 2: timing needs a big array - 6 numbers sort in ~0 ms
    srand(1);                                // fixed seed: same "random" data every run
    for (int i = 0; i < BIG; i++)
        original[i] = rand() % 100000;

    cout << "\nTime to sort " << BIG << " random numbers:" << endl;
    clock_t start, end;

    copyArray(original, work, BIG);
    start = clock();  quickSort(work, 0, BIG - 1);  end = clock();
    cout << "Quick Sort     : " << 1000.0 * (end - start) / CLOCKS_PER_SEC << " ms" << endl;

    copyArray(original, work, BIG);
    start = clock();  bubbleSort(work, BIG);  end = clock();
    cout << "Bubble Sort    : " << 1000.0 * (end - start) / CLOCKS_PER_SEC << " ms" << endl;

    copyArray(original, work, BIG);
    start = clock();  selectionSort(work, BIG);  end = clock();
    cout << "Selection Sort : " << 1000.0 * (end - start) / CLOCKS_PER_SEC << " ms" << endl;

    copyArray(original, work, BIG);
    start = clock();  insertionSort(work, BIG);  end = clock();
    cout << "Insertion Sort : " << 1000.0 * (end - start) / CLOCKS_PER_SEC << " ms" << endl;

    return 0;
}
```

SCREEN (REAL RUN)

```
Enter number of elements: 6
Enter elements of the array:
10 7 8 9 1 5
```

```
Quick Sort     : 1 5 7 8 9 10
Bubble Sort    : 1 5 7 8 9 10
Selection Sort : 1 5 7 8 9 10
Insertion Sort : 1 5 7 8 9 10

Time to sort 10000 random numbers:
Quick Sort     : 1 ms
Bubble Sort    : 114 ms
Selection Sort : 53 ms
Insertion Sort : 37 ms
```

The times vary from PC to PC, but the **ranking** is what the experiment shows: quick sort is far faster than the three $O(n^2)$ sorts on large random input.

Algorithm	Best	Average	Worst	Extra space	Stable?
Quick sort	O(n log n)	O(n log n)	$O(n^2)$	O(log n) avg (recursion)	No
Bubble sort (with early exit)	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes
Selection sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	No
Insertion sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes
Merge sort (assignment)	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes

QUICK SORT'S WORST CASE, IN THIS PROGRAM

With the **last element as pivot**, an **already sorted** array is the worst case: every pivot is the largest, so each partition peels off just one element. That gives $n(n-1)/2$ comparisons (15 for $n = 6$; tested in 7.6). Fix: pick the middle (or a random) element as pivot.

WRITE IN YOUR RECORD

Aim: To sort an integer array using quick sort and compare its performance with bubble, selection and insertion sort.

Algorithm: quickSort(low, high): if low < high → pi = partition(low, high) → quickSort(low, pi−1) → quickSort(pi+1, high). partition: pivot = arr[high], i = low−1; for j = low…high−1: if arr[j] < pivot, i++ and swap arr[i], arr[j]; finally swap arr[i+1], arr[high]; return i+1. Comparison: copy the same input for each sort, time each with clock().

Complexity: best/average O(n log n), worst $O(n^2)$ (sorted input with last-element pivot); space O(log n) average for the recursion, O(n) worst.

Result: 10 7 8 9 1 5 → 1 5 7 8 9 10. All four sorts gave the same output. On 10 000 random numbers quick sort took about 1 ms; bubble, selection and insertion took tens to over a hundred ms (bubble slowest).

7.5 Viva questions

Question	Answer
What is divide and conquer?	Split into smaller subproblems, solve each (recursively), combine. Quick sort splits by a pivot.
What is a pivot?	The element used to split; after partition it sits at its final sorted index.
Why `low < high` in quickSort?	A part with 0 or 1 element is sorted. This is the recursion's stopping condition (base case).
Why O(n log n) on average?	Each level of recursion does O(n) partition work, and balanced splits give about $\log_2 n$ levels.
Worst case and when?	$O(n^2)$, when the pivot is always the smallest or largest (sorted / reverse-sorted input with last-element pivot).
Space complexity?	In-place, but the recursion stack uses O(log n) on average and O(n) in the worst case.
Is quick sort stable?	No. Long-distance swaps can reorder equal elements.
Why copy the array before each sort in the comparison?	So every algorithm sorts the same unsorted input; a fair test.
Why does `swapElements` use `&`?	To swap the actual array elements, not copies.

7.6 If the examiner changes the question

"Separate functions for input, partitioning, sorting and display" is the program above. The rest (7 checks, all pass):

Descending order (one character in partition):

```
// Descending: the ONLY change is  arr[j] < pivot  ->  arr[j] > pivot
int partitionDesc(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++)
        if (arr[j] > pivot)
        {
            i++;
            swapElements(arr[i], arr[j]);
        }
    swapElements(arr[i + 1], arr[high]);
    return i + 1;
}

void quickSortDesc(int arr[], int low, int high)
{
    if (low < high)
    {
        int pi = partitionDesc(arr, low, high);
        quickSortDesc(arr, low, pi - 1);
```

```
        quickSortDesc(arr, pi + 1, high);
    }
}
```

First element as pivot: swap it to the end, then partition exactly as before (`quickSort` stays the same, just calling `partitionFirst`):

```
// First element as pivot: swap it to the end, then partition exactly as before
int partitionFirst(int arr[], int low, int high)
{
    swapElements(arr[low], arr[high]);
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++)
        if (arr[j] < pivot)
        {
            i++;
            swapElements(arr[i], arr[j]);
        }
    swapElements(arr[i + 1], arr[high]);
    return i + 1;
}
```

Middle element as pivot (the worst-case fix):

```
// Middle element as pivot: same trick (sorted input no longer gives the worst case)
int partitionMiddle(int arr[], int low, int high)
{
    int mid = low + (high - low) / 2;
    swapElements(arr[mid], arr[high]);
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++)
        if (arr[j] < pivot)
        {
            i++;
            swapElements(arr[i], arr[j]);
        }
    swapElements(arr[i + 1], arr[high]);
    return i + 1;
}

void quickSortMiddle(int arr[], int low, int high)
{
    if (low < high)
    {
        int pi = partitionMiddle(arr, low, high);
        quickSortMiddle(arr, low, pi - 1);
        quickSortMiddle(arr, pi + 1, high);
    }
}
```

Count comparisons and swaps. On the manual's example this gives 11 comparisons and 5 swaps; on a sorted array of 6, 15 comparisons (the worst case). Run it on sorted, reverse-sorted and random input for the "compare performance" item.

```
// Global counters: every function can add to them; reset to 0 before sorting
int comparisons = 0, swaps = 0;

int partitionCount(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++)
    {
        comparisons++;
        if (arr[j] < pivot)
        {
            i++;
            swapElements(arr[i], arr[j]);
            swaps++;
        }
    }
    swapElements(arr[i + 1], arr[high]);
    swaps++;
    return i + 1;
}

void quickSortCount(int arr[], int low, int high)
{
    if (low < high)
    {
        int pi = partitionCount(arr, low, high);
        quickSortCount(arr, low, pi - 1);
        quickSortCount(arr, pi + 1, high);
    }
}
```

Merge sort (for "compare quick sort with merge sort"; time it with `clock()` as in 7.4):

```
// Merge Sort: split in half, sort each half, merge the two sorted halves
void merge(int arr[], int low, int mid, int high)
{
    int temp[100];                          // holds the merged result
    int i = low, j = mid + 1, k = 0;
    while (i <= mid && j <= high)
    {
        if (arr[i] <= arr[j])
            temp[k++] = arr[i++];
        else
            temp[k++] = arr[j++];
    }
    while (i <= mid)                        // leftovers of the left half
        temp[k++] = arr[i++];
    while (j <= high)                       // leftovers of the right half
        temp[k++] = arr[j++];
    for (int t = 0; t < k; t++)             // copy back
        arr[low + t] = temp[t];
}

void mergeSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int mid = (low + high) / 2;
```

```
        mergeSort(arr, low, mid);
        mergeSort(arr, mid + 1, high);
        merge(arr, low, mid, high);
    }
}
```

CHECK 7.1

Partition `4 9 2 7 5` (pivot 5). Track i and the array after each swap. What index does partition return?

CHECK 7.2

Without looking, write `quickSort(int arr[], int low, int high)`. Six lines.

Answers

Check 7.1: pivot 5, i = −1. j = 0: 4 < 5 → i = 0, swap(arr[0], arr[0]) → `4 9 2 7 5`. j = 1: 9 no. j = 2: 2 < 5 → i = 1, swap(arr[1], arr[2]) → `4 2 9 7 5`. j = 3: 7 no. End: swap(arr[2], arr[4]) → `4 2 5 7 9`. Returns **2**.

Check 7.2: `void quickSort(int arr[], int low, int high) { if (low < high) { int pi = partitionArray(arr, low, high); quickSort(arr, low, pi - 1); quickSort(arr, pi + 1, high); } }`

8 · Exp 9 (probable) — Stack using an array

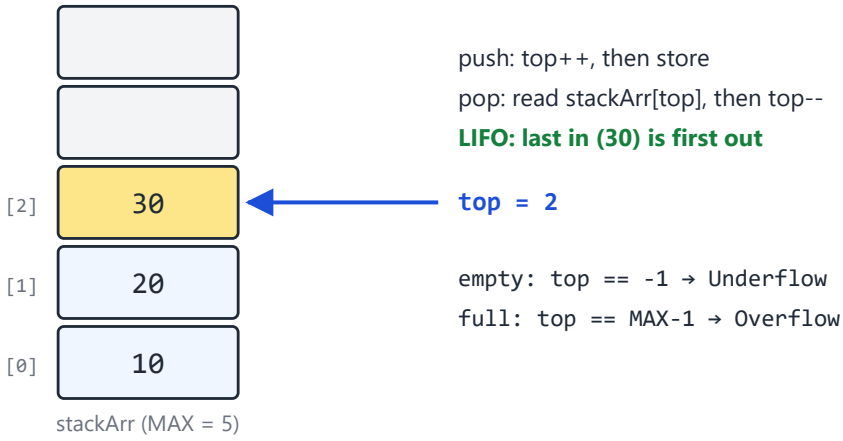
Not in your uploaded files. No Exp 9 file arrived, but Exp 10's figures are captioned "Enqueue Operation on Stack", which points to a stack experiment just before it. This chapter is written in exactly the style of Exp 10, so if Exp 9 is Stack, you're covered. Confirm with a classmate.

One open end (top) → push: top++, store → pop: read, top-- → LIFO

8.1 The idea

A **stack** is a pile: you add and remove only at the **top**. The last item pushed is the first popped: **LIFO** (Last In, First Out). Examples: undo history, the browser back button, function calls (the "call stack" quick sort's recursion uses).

> **GUESS FIRST**
> Push 10, 20, 30, then pop twice. Which values come out, in what order, and what is left?



An array stack after pushing 10, 20, 30. top is the index of the top element; −1 means empty.

8.2 The program

New C++ here (also used in Exp 10):

- `#define MAX 5`: the compiler replaces every `MAX` with 5. Change it in one place to resize.
- **Global variables** (declared above all functions): `stackArr` and `top` are shared by `push`, `pop`, `peek`, `display` and `main`.
- **Menu loop:** `do { show menu; read choice; act } while (choice != 5);` runs until the user picks Exit.

E09_STACK.CPP

```
#include <iostream>
using namespace std;

#define MAX 5
int stackArr[MAX];
int top = -1;                                   // -1 = empty
```

```
void push(int value)
{
    if (top == MAX - 1)
        cout << "Stack Overflow!" << endl;
    else
    {
        top++;
        stackArr[top] = value;
        cout << value << " pushed into stack." << endl;
    }
}

void pop()
{
    if (top == -1)
        cout << "Stack Underflow!" << endl;
    else
    {
        cout << stackArr[top] << " popped from stack." << endl;
        top--;
    }
}

void peek()
{
    if (top == -1)
        cout << "Stack is empty." << endl;
    else
        cout << "Top element is " << stackArr[top] << endl;
}

void display()
{
    if (top == -1)
        cout << "Stack is empty." << endl;
    else
    {
        cout << "Stack elements (top to bottom):" << endl;
        for (int i = top; i >= 0; i--)
            cout << stackArr[i] << " ";
        cout << endl;
    }
}

int main()
{
    int choice, value;

    do
    {
        cout << "\n1. Push" << endl;
        cout << "2. Pop" << endl;
        cout << "3. Peek" << endl;
        cout << "4. Display" << endl;
        cout << "5. Exit" << endl;
        cout << "Enter your choice: ";
        cin >> choice;

        if (choice == 1)
        {
            cout << "Enter value to push: ";
```

```
            cin >> value;
            push(value);
        }
        else if (choice == 2)
            pop();
        else if (choice == 3)
            peek();
        else if (choice == 4)
            display();
        else if (choice == 5)
            cout << "Program terminated." << endl;
        else
            cout << "Invalid choice!" << endl;
    } while (choice != 5);

    return 0;
}
```

Push 10, 20, 30 → display → peek → pop all three → pop again (underflow) → exit:

SCREEN (REAL RUN; MENU SHOWN ONCE, IT REPRINTS BEFORE EACH CHOICE)

```
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter value to push: 10
10 pushed into stack.
Enter your choice: 1
Enter value to push: 20
20 pushed into stack.
Enter your choice: 1
Enter value to push: 30
30 pushed into stack.
Enter your choice: 4
Stack elements (top to bottom):
30 20 10
Enter your choice: 3
Top element is 30
Enter your choice: 2
30 popped from stack.
Enter your choice: 4
Stack elements (top to bottom):
20 10
Enter your choice: 2
20 popped from stack.
Enter your choice: 2
10 popped from stack.
Enter your choice: 2
Stack Underflow!
Enter your choice: 5
Program terminated.
```

Overflow test (6 pushes into MAX = 5):

SCREEN (REAL RUN; MENU SHOWN ONCE, IT REPRINTS BEFORE EACH CHOICE)

```
1. Push
2. Pop
```

```
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter value to push: 1
1 pushed into stack.
Enter your choice: 1
Enter value to push: 2
2 pushed into stack.
Enter your choice: 1
Enter value to push: 3
3 pushed into stack.
Enter your choice: 1
Enter value to push: 4
4 pushed into stack.
Enter your choice: 1
Enter value to push: 5
5 pushed into stack.
Enter your choice: 1
Enter value to push: 6
Stack Overflow!
Enter your choice: 5
Program terminated.
```

WRITE IN YOUR RECORD

Aim: To implement a stack using an array with push, pop, peek and display operations.

Algorithm: top = −1. *Push:* if top == MAX−1 → overflow; else top++, stackArr[top] = value. *Pop:* if top == −1 → underflow; else print stackArr[top], top−−. *Peek:* print stackArr[top]. *Display:* print from top down to 0.

Complexity: push, pop, peek O(1); display O(n); space O(MAX).

Result: Pushed 10, 20, 30; popped 30, 20, 10 (LIFO). Overflow on the 6th push and underflow on popping an empty stack were detected.

8.3 Viva questions

Question	Answer
What is LIFO?	Last In, First Out: the most recently added item is removed first.
Overflow / underflow?	Push onto a full stack (top == MAX−1) / pop from an empty stack (top == −1).
Why does top start at −1?	So the first push makes it 0, the first array index. −1 = empty.
Stack vs. queue?	Stack: one end, LIFO. Queue: insert at rear, delete at front, FIFO.
Applications?	Function calls and recursion, undo, expression evaluation (postfix), bracket matching, backtracking.
Does pop erase the value?	No, it only moves top down; the old value is overwritten by the next push.

CHECK 8.1

MAX = 3. Operations: push 4, push 7, pop, push 9, push 2, push 6. What does each print? What is top at the end?

Answers

Check 8.1: 4 pushed (top 0), 7 pushed (top 1), pop prints 7 (top 0), 9 pushed (top 1), 2 pushed (top 2), push 6 → **Stack Overflow!** (top == MAX−1 = 2). top ends at **2**, and the stack is 4 9 2 (bottom → top).

9 · Exp 10 — Queue using an array (enqueue, dequeue)

A queue is a line at a ticket counter: join at the back (rear), leave from the front. Two indexes, front and rear, do all the work.

Two ends → enqueue: rear++ → dequeue: front++ → FIFO → linear-queue flaw → circular queue

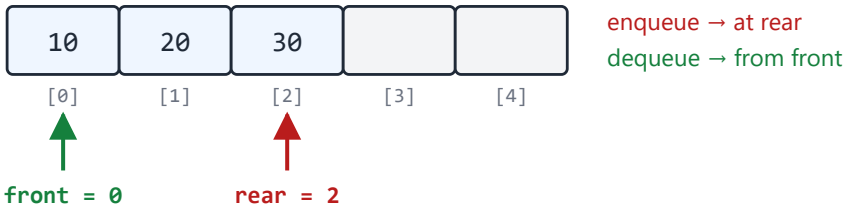
9.1 The idea

A **queue** inserts at the **rear** and removes from the **front**: **FIFO** (First In, First Out). `front` and `rear` both start at **−1** (empty).

GUESS FIRST

MAX = 5. Enqueue 1, 2, 3, 4, 5, dequeue twice, then enqueue 6. There are 2 empty slots. Does the program accept 6?

After enqueue 10, 20, 30 (MAX = 5)



After 1 dequeue:



The manual's example: enqueue 10, 20, 30, then one dequeue.

Operation	Check first	Then
enqueue(v)	`rear == MAX-1` → Overflow	if `front == -1` set `front = 0`; `rear++`; `queueArr[rear] = v`
dequeue()	`front == -1` → Underflow	print `queueArr[front]`; if `front == rear` (last one) reset both to −1, else `front++`
display()	`front == -1` → empty	print `queueArr[front]` … `queueArr[rear]`

WHY RESET TO −1 WHEN THE LAST ELEMENT LEAVES

After dequeuing the only element, front would pass rear, which is confusing. Resetting both to −1 means "empty" is always `front == -1`, and the next enqueue starts again from index 0.

9.2 The program (corrected)

The manual's **listing** (page 5) is right; its **screenshot** (page 9) doesn't compile. In the screenshot, `enqueue(value);` sits after the `if`'s closing brace, so the next `else` has no `if`; choice 3 prints "Program terminated" instead of displaying; and there's a bare `else` with no statement:

G++ SAYS

```
m10_screenshot.cpp:23:9: error: 'else' without a previous 'if'
   23 |         else if (choice == 2)
      |         ^~~~
m10_screenshot.cpp:25:9: error: expected '}' before 'else'
   25 |         else if (choice == 3)
      |         ^~~~
m10_screenshot.cpp:28:5: error: expected primary-expression before '}' token
   28 |     } while (choice != 4);
      |     ^
```

Two more fixes: the array is renamed from `queue` to `queueArr` (with `#include <bits/stdc++.h>`, `queue` clashes with the library's `std::queue`), and the dead `|| front > rear` test is dropped (after the reset rule it can never be true).

E10_QUEUE.CPP

```cpp
#include <iostream>
using namespace std;

#define MAX 5
int queueArr[MAX];         // FIX: renamed from "queue" - clashes with std::queue
int front = -1;
int rear = -1;

void enqueue(int value)
{
    if (rear == MAX - 1)
    {
        cout << "Queue Overflow!" << endl;
    }
    else
    {
        if (front == -1)                    // first element
            front = 0;
        rear++;
        queueArr[rear] = value;
        cout << value << " inserted into queue." << endl;
    }
}

void dequeue()
{
    if (front == -1)
    {
        cout << "Queue Underflow!" << endl;
    }
    else
    {
        cout << queueArr[front] << " deleted from queue." << endl;
        if (front == rear)                  // that was the last element
        {
```

```
            front = -1;
            rear = -1;
        }
        else
        {
            front++;
        }
    }
}

void display()
{
    if (front == -1)
    {
        cout << "Queue is empty." << endl;
    }
    else
    {
        cout << "Queue elements are:" << endl;
        for (int i = front; i <= rear; i++)
            cout << queueArr[i] << " ";
        cout << endl;
    }
}

int main()
{
    int choice, value;

    do
    {
        cout << "\n1. Enqueue" << endl;
        cout << "2. Dequeue" << endl;
        cout << "3. Display" << endl;
        cout << "4. Exit" << endl;
        cout << "Enter your choice: ";
        cin >> choice;

        if (choice == 1)
        {
            cout << "Enter value to enqueue: ";
            cin >> value;
            enqueue(value);
        }
        else if (choice == 2)
        {
            dequeue();
        }
        else if (choice == 3)
        {
            display();
        }
        else if (choice == 4)
        {
            cout << "Program terminated." << endl;
        }
        else
        {
            cout << "Invalid choice!" << endl;
        }
    } while (choice != 4);
```

```
    return 0;
}
```

The manual's Sample Inputs 1 and 2 in one run:

SCREEN (REAL RUN; MENU SHOWN ONCE, IT REPRINTS BEFORE EACH CHOICE)

```
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 10
10 inserted into queue.
Enter your choice: 1
Enter value to enqueue: 20
20 inserted into queue.
Enter your choice: 1
Enter value to enqueue: 30
30 inserted into queue.
Enter your choice: 3
Queue elements are:
10 20 30
Enter your choice: 2
10 deleted from queue.
Enter your choice: 3
Queue elements are:
20 30
Enter your choice: 4
Program terminated.
```

9.3 The linear queue's flaw (a favourite viva question)

Linear queue flaw: enqueue 1..5, dequeue twice, enqueue 7



so the program prints "Queue Overflow!" (verified run). The circular queue fixes this.

Slots freed by dequeue are never reused, because rear only moves forward.

SCREEN (REAL RUN; MENU SHOWN ONCE, IT REPRINTS BEFORE EACH CHOICE)

```
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 2
Queue Underflow!
Enter your choice: 1
Enter value to enqueue: 1
1 inserted into queue.
Enter your choice: 1
Enter value to enqueue: 2
2 inserted into queue.
```

```
Enter your choice: 1
Enter value to enqueue: 3
3 inserted into queue.
Enter your choice: 1
Enter value to enqueue: 4
4 inserted into queue.
Enter your choice: 1
Enter value to enqueue: 5
5 inserted into queue.
Enter your choice: 1
Enter value to enqueue: 6
Queue Overflow!
Enter your choice: 2
1 deleted from queue.
Enter your choice: 2
2 deleted from queue.
Enter your choice: 1
Enter value to enqueue: 7
Queue Overflow!
Enter your choice: 3
Queue elements are:
3 4 5
Enter your choice: 4
Program terminated.
```

Underflow on the empty queue; then 1–5 fill it; 6 overflows (correct, it's full); two dequeues free slots 0 and 1, **yet 7 still overflows**, because rear is still MAX−1. The fix is the **circular queue** (9.5).

WRITE IN YOUR RECORD

Aim: To implement a queue using an array and perform enqueue, dequeue and display operations.

Algorithm: front = rear = −1. *Enqueue:* if rear == MAX−1 → overflow; else (if front == −1, front = 0), rear++, queue[rear] = value. *Dequeue:* if front == −1 → underflow; else print queue[front]; if front == rear, front = rear = −1, else front++. *Display:* print queue[front…rear].

Complexity: enqueue O(1), dequeue O(1), display O(n); space O(MAX).

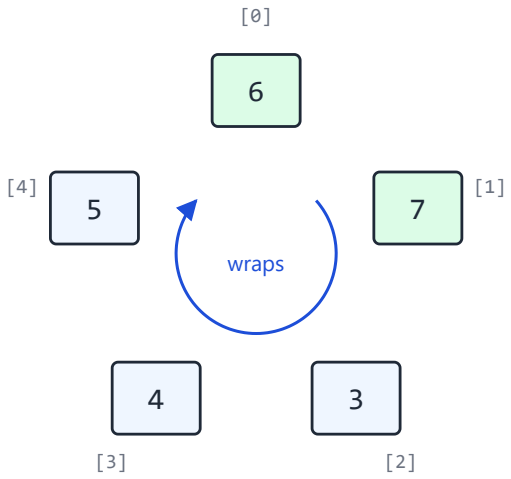
Result: Enqueued 10, 20, 30 (display 10 20 30); dequeue removed 10 (display 20 30), confirming FIFO. Overflow and underflow were detected.

9.4 Viva questions

Question	Answer
What is FIFO?	First In, First Out: elements leave in the order they arrived.
Where is insertion / deletion done?	Insertion at the rear, deletion at the front.
Overflow / underflow conditions?	rear == MAX−1 / front == −1.
Drawback of a linear array queue?	Freed slots at the front are never reused, so it can report overflow while slots are empty.
How does a circular queue fix it?	Indexes wrap: `rear = (rear + 1) % MAX`, so slot 0 is reused after slot MAX−1.
Applications?	CPU and printer scheduling, keyboard buffers, network packets, breadth-first search.
Why are enqueue and dequeue O(1)?	Each touches one index; nothing is shifted (unlike array deletion in Exp 4).

9.5 If the examiner changes the question

Circular queue (full program). The only changes from Exp 10: wrap with `% MAX`, and "full" becomes `(rear + 1) % MAX == front`.




```
front = 2 (value 3)
rear = 1 (value 7)

rear = (rear + 1) % MAX
4 → (4+1) % 5 = 0

full when (rear+1) % MAX == front
```

green = new values stored in the slots the linear queue wasted

After enqueue 1–5, dequeue 1 and 2, enqueue 6 and 7: the new values reuse slots 0 and 1.

E10B_CIRCULAR_QUEUE.CPP

```cpp
// Exp 10 assignment: Circular Queue using an array.
// Same as the linear queue, except rear and front wrap around with  % MAX.
#include <iostream>
using namespace std;

#define MAX 5
int cq[MAX];
int front = -1, rear = -1;
```

```
bool isEmpty() { return front == -1; }
bool isFull()  { return (rear + 1) % MAX == front; }   // next slot after rear is front

void enqueue(int value)
{
    if (isFull())
    {
        cout << "Queue Overflow!" << endl;
        return;
    }
    if (isEmpty())
        front = 0;
    rear = (rear + 1) % MAX;                 // 4 -> 0 : wrap around
    cq[rear] = value;
    cout << value << " inserted into queue." << endl;
}

void dequeue()
{
    if (isEmpty())
    {
        cout << "Queue Underflow!" << endl;
        return;
    }
    cout << cq[front] << " deleted from queue." << endl;
    if (front == rear)                       // last element removed
        front = rear = -1;
    else
        front = (front + 1) % MAX;
}

void display()
{
    if (isEmpty())
    {
        cout << "Queue is empty." << endl;
        return;
    }
    cout << "Queue elements are:" << endl;
    int i = front;
    while (true)
    {
        cout << cq[i] << " ";
        if (i == rear)
            break;
        i = (i + 1) % MAX;
    }
    cout << endl;
}

int main()
{
    int choice, value;
    do
    {
        cout << "\n1. Enqueue" << endl;
        cout << "2. Dequeue" << endl;
        cout << "3. Display" << endl;
        cout << "4. Exit" << endl;
        cout << "Enter your choice: ";
```

```
        cin >> choice;

        if (choice == 1)
        {
            cout << "Enter value to enqueue: ";
            cin >> value;
            enqueue(value);
        }
        else if (choice == 2)
            dequeue();
        else if (choice == 3)
            display();
        else if (choice == 4)
            cout << "Program terminated." << endl;
        else
            cout << "Invalid choice!" << endl;
    } while (choice != 4);

    return 0;
}
```

The same sequence that defeated the linear queue. 6 and 7 are now accepted:

SCREEN (REAL RUN; MENU SHOWN ONCE, IT REPRINTS BEFORE EACH CHOICE)

```
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 1
1 inserted into queue.
Enter your choice: 1
Enter value to enqueue: 2
2 inserted into queue.
Enter your choice: 1
Enter value to enqueue: 3
3 inserted into queue.
Enter your choice: 1
Enter value to enqueue: 4
4 inserted into queue.
Enter your choice: 1
Enter value to enqueue: 5
5 inserted into queue.
Enter your choice: 1
Enter value to enqueue: 6
Queue Overflow!
Enter your choice: 2
1 deleted from queue.
Enter your choice: 2
2 deleted from queue.
Enter your choice: 1
Enter value to enqueue: 6
6 inserted into queue.
Enter your choice: 1
Enter value to enqueue: 7
7 inserted into queue.
Enter your choice: 3
Queue elements are:
3 4 5 6 7
Enter your choice: 1
```

```
Enter value to enqueue: 8
Queue Overflow!
Enter your choice: 4
Program terminated.
```

Linear vs. circular queue ("WAP to compare"): run both programs on the input above. The linear one reports overflow with free slots; the circular one stores 6 and 7. Both are O(1) per operation.

Peek, isEmpty / isFull, show front and rear (add to Exp 10's program and to the menu):

```
void peek()
{
    if (front == -1)
        cout << "Queue is empty." << endl;
    else
        cout << "Front element is " << queueArr[front] << endl;   // look, don't remove
}
```

```
bool isEmpty() { return front == -1; }
bool isFull()  { return rear == MAX - 1; }
```

```
void showFrontRear()
{
    if (front == -1)
        cout << "Queue is empty." << endl;
    else
        cout << "Front = " << queueArr[front] << ", Rear = " << queueArr[rear] << endl;
}
```

Reverse the queue (swap the two ends, move inwards):

```
// Reverse the elements between front and rear: swap the two ends, move inwards
void reverseQueue()
{
    if (front == -1)
        return;
    int i = front, j = rear;
    while (i < j)
    {
        int temp = queueArr[i];
        queueArr[i] = queueArr[j];
        queueArr[j] = temp;
        i++;
        j--;
    }
}
```

Queue using a linked list (read chapter 10 first; no MAX, so no overflow):

```
// Queue using a linked list: no fixed size, so no overflow.
// Enqueue at the rear end of the list, dequeue from the front end.
struct QNode
{
    int data;
    QNode *next;
};
```

```
QNode *qFront = NULL, *qRear = NULL;

void enqueueLL(int value)
{
    QNode *newNode = new QNode;
    newNode->data = value;
    newNode->next = NULL;
    if (qRear == NULL)                      // empty queue
        qFront = qRear = newNode;
    else
    {
        qRear->next = newNode;
        qRear = newNode;
    }
}

int dequeueLL()                             // returns -1 if empty
{
    if (qFront == NULL)
    {
        cout << "Queue Underflow!" << endl;
        return -1;
    }
    QNode *temp = qFront;
    int value = temp->data;
    qFront = qFront->next;
    if (qFront == NULL)                     // queue became empty
        qRear = NULL;
    delete temp;
    return value;
}
```

Queue for CPU scheduling (FCFS). Processes are served in arrival order. Tested: bursts 24, 3, 3 give waiting times 0, 24, 27, average 17.

```
// CPU scheduling (First Come First Served): processes wait in a queue in arrival order.
// Waiting time of a process = sum of burst times of everyone before it.
float fcfs(int burst[], int n)
{
    int waiting = 0, totalWait = 0;
    cout << "Process\tBurst\tWaiting\tTurnaround" << endl;     // \t = tab
    for (int i = 0; i < n; i++)             // dequeue in FIFO order
    {
        int turnaround = waiting + burst[i];
        cout << "P" << i + 1 << "\t" << burst[i] << "\t" << waiting << "\t" << turnaround << endl;
        totalWait += waiting;
        waiting += burst[i];
    }
    cout << "Average waiting time = " << (float)totalWait / n << endl;
    return (float)totalWait / n;
}
```

CHECK 9.1

MAX = 4, linear queue. enqueue 5, enqueue 6, dequeue, dequeue, enqueue 7. What are front and rear after each step?

CHECK 9.2

Circular queue, MAX = 4, front = 2, rear = 3. Enqueue 9: which index is it stored at? Is the queue full now?

Answers

Check 9.1: enqueue 5 → front 0, rear 0. enqueue 6 → 0, 1. dequeue (5) → 1, 1. dequeue (6): front == rear → reset → −1, −1. enqueue 7 → 0, 0.

Check 9.2: rear = (3 + 1) % 4 = **0**, so 9 goes to index 0. Full? (0 + 1) % 4 = 1 ≠ front (2), so **not full**: index 1 is still free.

10 · Exp 11 — Singly linked list (insert, delete, traverse)

The pointers from Exp 1 finally pay off. New C++: struct, new / delete, and the arrow operator ->. Read 10.1 slowly; everything after builds on it.

struct Node → new Node → head → insert at end → delete from beginning → traverse until NULL

10.1 New C++: struct, new, delete, ->

A **struct** bundles several variables into one new type:

```
struct Node
{
    int data;      // the value
    Node *next;    // address of the next node (NULL if none)
};
```

- `new Node` asks for memory for one Node **while the program runs** and gives back its **address**: `Node *newNode = new Node;`
- `p->data` means "the `data` field of the node that `p` points to". It is shorthand for `(*p).data`.
- `delete p;` gives that node's memory back. Every `new` should have a matching `delete`.
- `NULL` is the "points nowhere" address: the end of the list, or an empty list (`head == NULL`).

GUESS FIRST

An array keeps its elements side by side. How does a linked list know where its second element is, if the nodes can be anywhere in memory?

HEAD → 10 → 20 → 30 → NULL



Insert 40 at end: walk temp to the node whose next is NULL, then

```
temp->next = newNode;
```

Delete from beginning:

```
temp = head; head = head->next; delete temp;
```

Order matters: move head BEFORE delete, or the rest of the list is lost.

The manual's list after inserting 10, 20, 30. Each node stores the address of the next.

The list is reached only through `head`, the address of the first node. Lose `head` and the whole list is lost.

10.2 The three operations

Operation	Steps	Time
insert at end	make node (next = NULL) → if `head == NULL`, head = node → else walk `temp` until `temp->next == NULL`, then `temp->next = newNode`	O(n) (the walk)
delete from beginning	if empty → message → else `temp = head`, `head = head->next`, `delete temp`	O(1)
traverse	`temp = head`; while `temp != NULL`: print `temp->data`, `temp = temp->next`	O(n)

THE TWO CLASSIC LINKED-LIST BUGS

1. **Forgetting `temp = temp->next;`** in traverse: `temp` never moves, so the loop never ends. The manual's IMPLEMENTATION screenshot (page 9) has exactly this bug, and its insert never links the new node in. Its **listing** (page 6) is correct; use that.
2. **`delete head;` before moving head**: the second node's address was inside the deleted node, so the rest of the list is lost. Always save the node in `temp`, move `head`, *then* `delete temp`.

10.3 The program

The manual's listing was correct (one line was cut off at the page edge: `cout << temp->data << " deleted from list." << endl;`).

E11_SINGLY_LINKED_LIST.CPP

```
#include <iostream>
using namespace std;

struct Node
{
    int data;
    Node *next;
};

Node *head = NULL;                          // empty list

// Insert at the end
void insert(int value)
{
    Node *newNode = new Node;               // ask for memory for one node
    newNode->data = value;
    newNode->next = NULL;

    if (head == NULL)
    {
        head = newNode;
    }
    else
    {
        Node *temp = head;
        while (temp->next != NULL)          // walk to the last node
            temp = temp->next;
```

```
        temp->next = newNode;
    }
    cout << value << " inserted into list." << endl;
}

// Delete from the beginning
void deleteNode()
{
    if (head == NULL)
    {
        cout << "List is empty." << endl;
    }
    else
    {
        Node *temp = head;
        cout << temp->data << " deleted from list." << endl;
        head = head->next;                  // move head first...
        delete temp;                        // ...then free the old first node
    }
}

void traverse()
{
    if (head == NULL)
    {
        cout << "List is empty." << endl;
    }
    else
    {
        Node *temp = head;
        cout << "Linked List: ";
        while (temp != NULL)
        {
            cout << temp->data << " ";
            temp = temp->next;              // without this line: infinite loop
        }
        cout << endl;
    }
}

int main()
{
    int choice, value;

    do
    {
        cout << "\n1. Insert" << endl;
        cout << "2. Delete" << endl;
        cout << "3. Traverse" << endl;
        cout << "4. Exit" << endl;
        cout << "Enter your choice: ";
        cin >> choice;

        if (choice == 1)
        {
            cout << "Enter value to insert: ";
            cin >> value;
            insert(value);
        }
        else if (choice == 2)
        {
```

```
            deleteNode();
        }
        else if (choice == 3)
        {
            traverse();
        }
        else if (choice == 4)
        {
            cout << "Program terminated." << endl;
        }
        else
        {
            cout << "Invalid choice!" << endl;
        }
    } while (choice != 4);

    return 0;
}
```

The manual's Sample Inputs 1 and 2 in one run:

SCREEN (REAL RUN; MENU SHOWN ONCE, IT REPRINTS BEFORE EACH CHOICE)

```
1. Insert
2. Delete
3. Traverse
4. Exit
Enter your choice: 1
Enter value to insert: 10
10 inserted into list.
Enter your choice: 1
Enter value to insert: 20
20 inserted into list.
Enter your choice: 1
Enter value to insert: 30
30 inserted into list.
Enter your choice: 3
Linked List: 10 20 30
Enter your choice: 2
10 deleted from list.
Enter your choice: 3
Linked List: 20 30
Enter your choice: 4
Program terminated.
```

Edge cases (delete and traverse on an empty list, delete the only node):

SCREEN (REAL RUN; MENU SHOWN ONCE, IT REPRINTS BEFORE EACH CHOICE)

```
1. Insert
2. Delete
3. Traverse
4. Exit
Enter your choice: 2
List is empty.
Enter your choice: 3
List is empty.
Enter your choice: 1
Enter value to insert: 5
5 inserted into list.
Enter your choice: 2
```

```
5 deleted from list.
Enter your choice: 3
List is empty.
Enter your choice: 4
Program terminated.
```

WRITE IN YOUR RECORD

Aim: To implement a singly linked list and perform insertion, deletion and traversal.

Algorithm: head = NULL. *Insert (end):* create node (data = value, next = NULL); if head == NULL, head = node; else move temp to the last node (temp->next == NULL) and set temp->next = node. *Delete (beginning):* if head == NULL → "empty"; else temp = head, head = head->next, delete temp. *Traverse:* temp = head; while temp ≠ NULL print temp->data, temp = temp->next.

Complexity: insert at end O(n), delete at beginning O(1), traverse O(n); space O(n) for n nodes.

Result: Inserted 10, 20, 30 (list: 10 20 30); deleting from the beginning removed 10 (list: 20 30).

10.4 Viva questions

Question	Answer
Linked list vs. array?	List: nodes anywhere in memory, size grows/shrinks, insert/delete by changing pointers, but no direct `a[i]` access (must walk) and an extra pointer per node. Array: contiguous, fixed size, O(1) indexing, O(n) insert/delete.
What does the last node's next hold?	NULL.
What is head?	A pointer to the first node; `head == NULL` means the list is empty.
Why is insert at end O(n)?	You must walk from head to the last node. (Keeping a `tail` pointer makes it O(1).)
Why is delete at beginning O(1)?	Only head changes; no walking.
What is `->`?	Access a field through a pointer: `p->data` = `(*p).data`.
Why `delete temp`?	Return the node's memory; otherwise it leaks.
Can you traverse backwards?	Not in a singly linked list (only next pointers). A doubly linked list adds `prev`.

10.5 If the examiner changes the question

Insert at end and delete from beginning are the experiment. The rest (9 checks, all pass), each a function to add above `main()` and to the menu:

Insert at the beginning (O(1)):

```
void insertBegin(int value)
{
    Node *newNode = new Node;
    newNode->data = value;
    newNode->next = head;                    // new node points to the old first node
    head = newNode;                          // and becomes the first node
}
```

Delete from the end. Stop at the second-last node:

```
void deleteEnd()
{
    if (head == NULL)
    {
        cout << "List is empty." << endl;
        return;
    }
    if (head->next == NULL)                  // only one node
    {
        cout << head->data << " deleted from list." << endl;
        delete head;
        head = NULL;
        return;
    }
    Node *temp = head;
    while (temp->next->next != NULL)         // stop at the SECOND-last node
        temp = temp->next;
    cout << temp->next->data << " deleted from list." << endl;
    delete temp->next;
    temp->next = NULL;                       // it is now the last node
}
```

Search (returns the position) and **count nodes**:

```
int search(int key)                          // 1-based position, -1 if absent
{
    int pos = 1;
    Node *temp = head;
    while (temp != NULL)
    {
        if (temp->data == key)
            return pos;
        temp = temp->next;
        pos++;
    }
    return -1;
}
```

```
int countNodes()
{
    int count = 0;
    for (Node *temp = head; temp != NULL; temp = temp->next)
        count++;
    return count;
}
```

Display in reverse (recursion: the deepest call prints first):

```
// Recursion: first print the REST of the list in reverse, then this node
void displayReverse(Node *p)
{
    if (p == NULL)
        return;
    displayReverse(p->next);
    cout << p->data << " ";
}
// call it as:  displayReverse(head);
```

Insert at a position:

```
void insertAtPos(int pos, int value)         // pos 1 = new first node
{
    if (pos < 1)
    {
        cout << "Invalid position." << endl;
        return;
    }
    if (pos == 1)
    {
        insertBegin(value);
        return;
    }
    Node *temp = head;
    for (int i = 1; i < pos - 1 && temp != NULL; i++)   // go to node pos-1
        temp = temp->next;
    if (temp == NULL)
    {
        cout << "Invalid position." << endl;
        return;
    }
    Node *newNode = new Node;
    newNode->data = value;
    newNode->next = temp->next;              // 1) new node points ahead
    temp->next = newNode;                    // 2) then link it in (order matters!)
}
```

Delete at a position:

```
void deleteAtPos(int pos)
{
    if (head == NULL || pos < 1)
    {
        cout << "Invalid position." << endl;
        return;
    }
    Node *del;
    if (pos == 1)
    {
        del = head;
        head = head->next;
    }
    else
    {
        Node *temp = head;
        for (int i = 1; i < pos - 1 && temp != NULL; i++)   // go to node pos-1
            temp = temp->next;
```

```
        if (temp == NULL || temp->next == NULL)
        {
            cout << "Invalid position." << endl;
            return;
        }
        del = temp->next;
        temp->next = del->next;              // bypass the node
    }
    cout << del->data << " deleted from list." << endl;
    delete del;
}
```

CHECK 10.1

For inserting a new node after node `temp`, these two lines come in some order: `temp->next = newNode;` and `newNode->next = temp->next;`. Which must come first, and what happens if you swap them?

CHECK 10.2

List: head → 4 → 8 → NULL. Trace `deleteNode()` then `insert(5)`. Draw the list after each.

Answers

Check 10.1: `newNode->next = temp->next;` **first**. If `temp->next = newNode;` runs first, the old `temp->next` address is overwritten, so `newNode->next` would point to newNode itself and the rest of the list is lost.

Check 10.2: delete: temp = (4), head = (8), delete temp → head → 8 → NULL. insert(5): walk to 8 (next is NULL), link → head → 8 → 5 → NULL.

11 · Exp 12 — Circular linked list (insert, delete, traverse)

Exp 11 with one change: the last node points back to head instead of NULL. Every "== NULL" test becomes "== head".

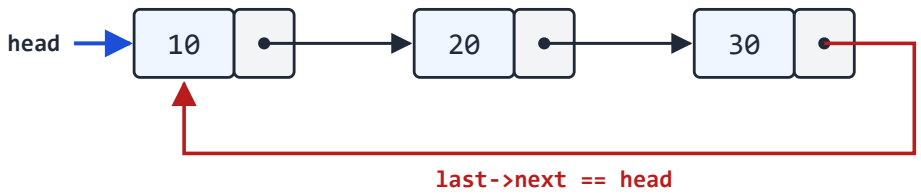
Exp 11 list → **last node points to head** → find last: next == head → traverse: do-while → delete first: fix the last node

11.1 The idea

GUESS FIRST

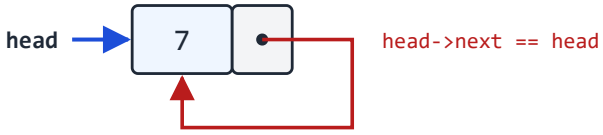
In a circular list there is no NULL to stop at. If traverse used `while (temp != head)` with `temp = head` at the start, what would it print?

Circular: the last node points back to head (never NULL)



Traverse with do { ... } while (temp != head); - a plain while (temp != head) would stop before printing anything.

The manual's list after inserting 10, 20, 30.



A single node points to itself. The program must treat this case separately.

	Singly (Exp 11)	Circular (Exp 12)
last node's next	NULL	head
"is this the last node?"	`temp->next == NULL`	`temp->next == head`
traverse loop	`while (temp != NULL)`	`do { … } while (temp != head);`
delete from beginning	O(1)	**O(n)**: the last node must be found and re-pointed to the new head

The answer to the guess: **nothing**. temp starts *equal* to head, so a `while` loop exits before its first pass. A `do … while` prints first and tests afterwards.

11.2 The program (corrected)

The manual's PROGRAM listing doesn't compile:

G++ SAYS

```
m12_listing.cpp:62:17: error: missing terminating " character
   62 |     {   cout << "\n1. Insert
      |                 ^
m12_listing.cpp:13:6: error: variable or field 'insert' declared void
   13 | void insert(intvalue)
      |      ^~~~~~
m12_listing.cpp:13:13: error: 'intvalue' was not declared in this scope
   13 | void insert(intvalue)
      |             ^~~~~~~~
m12_listing.cpp:46:15: error: unused variable 'temp' [-Werror=unused-variable]
   46 |     {   Node* temp = head;
      |               ^~~~
m12_listing.cpp:55:16: error: 'temp' was not declared in this scope; did you mean 'tm'?
   55 |     }   delete temp;
      |                ^~~~
m12_listing.cpp:62:14: error: no match for 'operator<<' (operand types are 'std::ostream' {aka
'std::basic_ostream<char>'} and 'std::ostream' {aka 'std::basic_ostream<char>'})
   62 |     {   cout << "\n1. Insert
      |         ~~~~ ^~
```

The four problems: `insert(intvalue)` (missing space); the menu `cout` lines are missing their closing `" << endl;` ; `delete temp;` sits after the closing `}` of the block where `temp` was declared; and `main()` is cut off at the page end. The IMPLEMENTATION screenshot (page 7) is correct and complete; this pack follows it.

E12_CIRCULAR_LINKED_LIST.CPP

```cpp
#include <iostream>
using namespace std;

struct Node
{
    int data;
    Node *next;
};

Node *head = NULL;

// Insert at the end
void insert(int value)                    // FIX: manual printed "intvalue"
{
    Node *newNode = new Node;
    newNode->data = value;

    if (head == NULL)
    {
        head = newNode;
        newNode->next = head;             // one node points to itself
    }
    else
    {
        Node *temp = head;
```

```
        while (temp->next != head)          // last node is the one pointing to head
            temp = temp->next;
        temp->next = newNode;
        newNode->next = head;               // close the circle
    }
    cout << value << " inserted into list." << endl;
}

// Delete from the beginning
void deleteNode()
{
    if (head == NULL)
    {
        cout << "List is empty." << endl;
    }
    else if (head->next == head)            // only one node
    {
        cout << head->data << " deleted from list." << endl;
        delete head;
        head = NULL;
    }
    else
    {
        Node *temp = head;
        Node *last = head;
        while (last->next != head)          // find the last node
            last = last->next;

        cout << head->data << " deleted from list." << endl;
        head = head->next;
        last->next = head;                  // last now points to the new head
        delete temp;                        // FIX: was outside this block (temp unknown there)
    }
}

void traverse()
{
    if (head == NULL)
    {
        cout << "List is empty." << endl;
    }
    else
    {
        Node *temp = head;
        cout << "Circular Linked List: ";
        do                                  // do-while: print head before testing
        {
            cout << temp->data << " ";
            temp = temp->next;
        } while (temp != head);
        cout << endl;
    }
}

int main()
{
    int choice, value;

    do
    {
        cout << "\n1. Insert" << endl;       // FIX: manual's menu strings were unclosed
```

```
        cout << "2. Delete" << endl;
        cout << "3. Traverse" << endl;
        cout << "4. Exit" << endl;
        cout << "Enter your choice: ";
        cin >> choice;

        if (choice == 1)
        {
            cout << "Enter value to insert: ";
            cin >> value;
            insert(value);
        }
        else if (choice == 2)
        {
            deleteNode();
        }
        else if (choice == 3)
        {
            traverse();
        }
        else if (choice == 4)
        {
            cout << "Program terminated." << endl;
        }
        else
        {
            cout << "Invalid choice!" << endl;
        }
    } while (choice != 4);

    return 0;
}
```

The manual's sample run, reproduced exactly:

SCREEN (REAL RUN; MENU SHOWN ONCE, IT REPRINTS BEFORE EACH CHOICE)

```
1. Insert
2. Delete
3. Traverse
4. Exit
Enter your choice: 1
Enter value to insert: 10
10 inserted into list.
Enter your choice: 1
Enter value to insert: 20
20 inserted into list.
Enter your choice: 1
Enter value to insert: 30
30 inserted into list.
Enter your choice: 3
Circular Linked List: 10 20 30
Enter your choice: 2
10 deleted from list.
Enter your choice: 3
Circular Linked List: 20 30
Enter your choice: 4
Program terminated.
```

Deleting the only node (the `head->next == head` branch), then deleting from an empty list:

SCREEN (REAL RUN; MENU SHOWN ONCE, IT REPRINTS BEFORE EACH CHOICE)

```
1. Insert
2. Delete
3. Traverse
4. Exit
Enter your choice: 1
Enter value to insert: 7
7 inserted into list.
Enter your choice: 2
7 deleted from list.
Enter your choice: 3
List is empty.
Enter your choice: 2
List is empty.
Enter your choice: 4
Program terminated.
```

DELETE-FROM-BEGINNING, THREE CASES

1. **Empty** (`head == NULL`): print a message.
2. **One node** (`head->next == head`): delete it, set `head = NULL` . Without this case, the general code would set head to the deleted node itself.
3. **General:** find `last` (its next is head), move `head = head->next` , set `last->next = head` , then `delete temp` .

WRITE IN YOUR RECORD

Aim: To implement a circular linked list and perform insertion, deletion and traversal.

Algorithm: *Insert (end):* create node; if head == NULL: head = node, node->next = head; else move temp until temp->next == head, temp->next = node, node->next = head. *Delete (beginning):* if empty → message; if one node → delete it, head = NULL; else find last, temp = head, head = head->next, last->next = head, delete temp. *Traverse:* temp = head; do { print temp->data; temp = temp->next } while (temp ≠ head).

Complexity: insert at end O(n), delete at beginning O(n) (finding the last node), traverse O(n); space O(n), auxiliary O(1).

Result: Inserted 10, 20, 30 (list: 10 20 30); deleted 10 (list: 20 30). The last node always pointed back to head.

11.3 Viva questions

Question	Answer
What is a circular linked list?	A linked list whose last node points to the first node instead of NULL.
How do you detect the last node?	`temp->next == head`.
Why `do-while` for traversal?	The stop condition (`temp == head`) is already true at the start; do-while prints head first.
Why is delete-from-beginning O(n) here but O(1) in Exp 11?	The last node's next must be updated to the new head, and finding it takes a walk. (A `tail` pointer would make it O(1).)
One-node list: what is head->next?	head itself.
Advantages?	Any node can reach every other node; natural for round-robin scheduling, multiplayer turns, repeating playlists.
Risk?	Infinite loops, if a traversal's stop condition is wrong.
Types?	Circular singly linked (next only) and circular doubly linked (next and prev).

11.4 If the examiner changes the question

The same assignment list as Exp 11, adapted so the circle stays closed (10 checks, all pass, each including "is it still circular?").

Insert at the beginning. The last node must point to the new head:

```
// Like insert at end, but afterwards head moves to the new node
void insertBegin(int value)
{
    Node *newNode = new Node;
    newNode->data = value;
    if (head == NULL)
    {
        head = newNode;
        newNode->next = head;
        return;
    }
    Node *last = head;
    while (last->next != head)              // the last node must point to the new first node
        last = last->next;
    newNode->next = head;
    last->next = newNode;
    head = newNode;
}
```

Delete from the end:

```
void deleteEnd()
{
    if (head == NULL)
    {
```

```
        cout << "List is empty." << endl;
        return;
    }
    if (head->next == head)                 // only one node
    {
        cout << head->data << " deleted from list." << endl;
        delete head;
        head = NULL;
        return;
    }
    Node *temp = head;
    while (temp->next->next != head)        // stop at the second-last node
        temp = temp->next;
    cout << temp->next->data << " deleted from list." << endl;
    delete temp->next;
    temp->next = head;                      // new last node closes the circle
}
```

Search and count (do-while again):

```
int search(int key)                         // 1-based position, -1 if absent
{
    if (head == NULL)
        return -1;
    Node *temp = head;
    int pos = 1;
    do
    {
        if (temp->data == key)
            return pos;
        temp = temp->next;
        pos++;
    } while (temp != head);
    return -1;
}
```

```
int countNodes()
{
    if (head == NULL)
        return 0;
    int count = 0;
    Node *temp = head;
    do
    {
        count++;
        temp = temp->next;
    } while (temp != head);
    return count;
}
```

Display in reverse (recursion that stops when the next node is head):

```
// Recursion that stops when it comes back round to head
void reverseFrom(Node *p)
{
    if (p->next != head)
        reverseFrom(p->next);
    cout << p->data << " ";
}

void displayReverse()
{
    if (head == NULL)
        cout << "List is empty.";
    else
        reverseFrom(head);
    cout << endl;
}
```

Insert / delete at a position:

```
void insertAtPos(int pos, int value)         // pos 1 = new first node
{
    int n = countNodes();
    if (pos < 1 || pos > n + 1)
    {
        cout << "Invalid position." << endl;
        return;
    }
    if (pos == 1)
    {
        insertBegin(value);
        return;
    }
    Node *temp = head;
    for (int i = 1; i < pos - 1; i++)        // go to node pos-1
        temp = temp->next;
    Node *newNode = new Node;
    newNode->data = value;
    newNode->next = temp->next;              // for pos = n+1 this is head: circle stays closed
    temp->next = newNode;
}
```

```
void deleteAtPos(int pos)
{
    int n = countNodes();
    if (pos < 1 || pos > n)
    {
        cout << "Invalid position." << endl;
        return;
    }
    if (pos == 1)                            // same as Exp 12's deleteNode()
    {
        Node *last = head;
        while (last->next != head)
            last = last->next;
        Node *del = head;
        cout << del->data << " deleted from list." << endl;
        if (n == 1)
            head = NULL;
        else
```

```
        {
            head = head->next;
            last->next = head;
        }
        delete del;
        return;
    }
    Node *temp = head;
    for (int i = 1; i < pos - 1; i++)
        temp = temp->next;
    Node *del = temp->next;
    temp->next = del->next;
    cout << del->data << " deleted from list." << endl;
    delete del;
}
```

CHECK 11.1

Circular list head → 3 → 6 → 9 → (back to 3). Run delete-from-beginning by hand: what are `last`, the new head, and `last->next` afterwards?

CHECK 11.2

Why does `insert` in Exp 12 need `newNode->next = head;` even in the `head == NULL` branch?

Answers

Check 11.1: last = node 9; head moves to node 6; `last->next = head` → 9 now points to 6; node 3 is deleted. List: 6 → 9 → (back to 6).

Check 11.2: A lone node is both first and last, so its next must point to head (itself) to keep the list circular. If it were left unset, the next traversal or insert would follow a garbage address.

12 · Exam day — procedure and one-page summary

Read this the night before and once more outside the lab. It doesn't replace typing the programs from memory (chapter 13).

12.1 The 6-step routine

1. **Read the question twice.** Is it the plain experiment or an ASSIGNMENT twist (descending order, insert at beginning, circular queue…)? Find the matching "If the examiner changes the question" section in your head.
2. **Write the record first** (aim, algorithm, complexity). It takes 5 minutes, earns marks even if the code fights you, and plans your program.
3. **Type the skeleton, compile it, then grow it.** `#include <iostream>` / `using namespace std;` / `int main() { return 0; }`. Build once (F9 / F11) so you know the setup works. Then add one function at a time and compile after each.
4. **Fix errors top-down.** Fix only the first error and recompile (chapter 1.6).
5. **Run the manual's sample input** and check the output against the expected answer below.
6. **Test one edge case** (empty list, bad position, overflow). Examiners like seeing it handled.

12.2 Expected outputs (the manual's sample inputs)

Exp	Input	Output
1	—	Value = 25, address = pointer value, value via pointer = 25
2	—	10 → 20 → 10 → 30; addresses base, +4, base, +8
3	10 20 30 40 50; begin 5 / pos 3 val 25 / end 60	5 10 20 30 40 50 / 10 20 25 30 40 50 / 10 20 30 40 50 60
4	10 20 30 40 50; begin / pos 3 / end	20 30 40 50 / 10 20 40 50 / 10 20 30 40
5	[[1,2],[3,4]] × [[5,6],[7,8]]	19 22 / 43 50
6	[[1,2,3],[4,5,6]]	1 4 / 2 5 / 3 6
7	5 2 4 6 1 3	1 2 3 4 5 6
8	10 7 8 9 1 5	1 5 7 8 9 10
9	push 10 20 30, pop	pops 30 (LIFO)
10	enqueue 10 20 30, dequeue	display 10 20 30; deletes 10; then 20 30
11	insert 10 20 30, delete	10 20 30; deletes 10; then 20 30
12	insert 10 20 30, delete	10 20 30; deletes 10; then 20 30

12.3 Complexity cheat sheet

Operation	Time	Space
Pointer access / arithmetic (Exp 1–2)	O(1)	O(1)
Array insert / delete at beginning or position (Exp 3–4)	O(n)	O(1)
Array insert / delete at end	O(1)	O(1)
Matrix multiply (r1·c1·c2) / n × n	$O(n^3)$	$O(n^2)$
Transpose m × n	O(m·n)	O(m·n)
Insertion sort: best / avg / worst	O(n) / $O(n^2)$ / $O(n^2)$	O(1)
Quick sort: best / avg / worst	O(n log n) / O(n log n) / $O(n^2)$	O(log n) avg, O(n) worst
Bubble / selection sort (worst)	$O(n^2)$	O(1)
Merge sort (all cases)	O(n log n)	O(n)
Stack push / pop / peek	O(1)	O(MAX)
Queue enqueue / dequeue; display	O(1); O(n)	O(MAX)
Singly list: insert end / delete begin / traverse	O(n) / O(1) / O(n)	O(n)
Circular list: insert end / delete begin / traverse	O(n) / O(n) / O(n)	O(n)

12.4 The lines that decide each program

Exp	The line(s) that matter
1	`ptr = &number;` · `*ptr`
2	`int *ptr = arr;` · `ptr++` moves one int (4 bytes)
3	`for (i = n; i >= pos; i--) arr[i] = arr[i-1];` · `arr[pos-1] = value;` · `n++`
4	`for (i = pos-1; i < n-1; i++) arr[i] = arr[i+1];` · `n--`
5	`C[i][j] = 0;` then `C[i][j] += A[i][k] * B[k][j];` for k < c1 · check `c1 == r2`
6	`T[j][i] = A[i][j];` · display `(T, cols, rows)`
7	`while (j >= 0 && arr[j] > key) { arr[j+1] = arr[j]; j--; }` · `arr[j+1] = key;`
8	`if (arr[j] < pivot) { i++; swap(arr[i], arr[j]); }` · `swap(arr[i+1], arr[high]); return i+1;`
9	full: `top == MAX-1` · empty: `top == -1`
10	full: `rear == MAX-1` · empty: `front == -1` · `if (front == rear) front = rear = -1;`
11	`while (temp->next != NULL)` · `head = head->next; delete temp;` · `temp = temp->next;`
12	`while (temp->next != head)` · `do { … } while (temp != head);` · `last->next = head;`

12.5 General viva questions

Question	Answer
What is a data structure?	A way of organising data in memory so operations on it are efficient (array, stack, queue, linked list, tree, graph).
Linear vs. non-linear?	Linear: elements in a sequence (array, stack, queue, list). Non-linear: hierarchy or network (tree, graph).
Static vs. dynamic?	Static: size fixed at compile time (array). Dynamic: grows at run time with `new` (linked list).
What does O(n) mean?	The running time grows in proportion to n (the input size), ignoring constants.
Time vs. space complexity?	Time = how the number of steps grows with n; space = how much extra memory grows with n.
Algorithm vs. program?	An algorithm is the step-by-step method (language-independent); a program is that method written in a language.
What is recursion?	A function calling itself on a smaller problem, with a base case that stops it (quick sort, merge sort, reverse display).
Why `using namespace std;`?	To use `cout`, `cin`, `endl` without writing `std::` each time.

13 · Revision set (the spaced re-test)

One question per key idea. Do this on paper, without the pack, a day or more after finishing the chapters, and again two or three days later. Typing the programs from memory is the real test.

> **HOW TO USE IT**
>
> Answer everything first, then mark with the answers at the end. Anything you missed: re-read that chapter section, then **type that program from scratch** the same day. Don't just re-read: being able to type it without looking is what earns marks.

Part A — type from memory (the actual exam skill)

Close the pack, open your editor, and type each program until it compiles and gives the expected output (chapter 12.2). Time yourself; aim for under 15 minutes each.

1. Exp 3 (insert, all three cases)
2. Exp 4 (delete, all three cases)
3. Exp 5 (matrix multiply with the three functions)
4. Exp 7 (insertion sort)
5. Exp 8 (quick sort with partition)
6. Exp 10 (queue with menu)
7. Exp 11 (singly linked list with menu)
8. Exp 12 (circular linked list with menu)

Part B — short questions

1. `int x = 9; int *p = &x; *p = 4;`. What is x now?
2. `int a[5]`; `int *p = a;`. Write the address of `a[3]` using p, and its value.
3. Insert at position 2 in `3 6 9` (n = 3), value 4. Final array?
4. Why does the insertion shift loop run from the end, but the deletion shift loop from the front?
5. Valid positions for insertion in an n-element array? For deletion?
6. A is 3 × 2, B is 2 × 4. Is A × B possible? Order of the result? Number of multiplications?
7. Transpose of a 4 × 1 matrix: what order?
8. Insertion sort on `3 1 2`: write the array after each pass.
9. Best-case input for insertion sort, and its time complexity?
10. Partition `7 2 9 4` with the last element as pivot. Final array and returned index?
11. Which input makes this program's quick sort $O(n^2)$?
12. Why does `swapElements` take `int &a, int &b`?
13. Linear queue, MAX = 3: enqueue 1, 2, 3, dequeue, enqueue 4. What happens, and why?
14. Circular queue condition for "full"?
15. Stack: push 1, 2, 3, pop, push 4, pop, pop. Which values come out?
16. In a singly linked list, how do you know you're at the last node? In a circular one?
17. What goes wrong if traverse forgets `temp = temp->next;`?
18. In delete-from-beginning, what goes wrong with `delete head; head = head->next;`?
19. Why is delete-from-beginning O(1) in a singly list but O(n) in the circular list (as programmed)?
20. Why is `int arr[n];` (n read with cin) a problem?
21. What does `p->data` mean, written another way?
22. Give the stability (stable or not) of insertion sort and of quick sort.

Answers

1. **4**: `*p = 4` writes through the pointer into x.
2. Address `p + 3`; value `*(p + 3)` (same as `a[3]`).
3. **3 4 6 9**.
4. In each direction, the element about to be overwritten must be copied first: shifting right, the last element moves first; shifting left, the one just after the gap moves first.
5. Insertion: 1 … n+1. Deletion: 1 … n.
6. Yes (2 = 2). Result 3 × 4. Multiplications: 3 · 4 · 2 = **24**.
7. **1 × 4**.
8. Pass 1 (key 1): `1 3 2`. Pass 2 (key 2): `1 2 3`.
9. An already sorted array: **O(n)** (n − 1 comparisons).
10. pivot 4, i = −1. j = 0: 7 no. j = 1: 2 < 4 → i = 0, swap(arr[0], arr[1]) → `2 7 9 4`. j = 2: 9 no. Final swap(arr[1], arr[3]) → **`2 4 9 7`**, returns **1**.
11. An already sorted (or reverse-sorted) array: the last-element pivot is always the largest (or smallest).
12. So the function swaps the real array elements, not copies.
13. **Overflow** on enqueue 4: rear is still MAX − 1 even though slot 0 was freed. This is the linear-queue flaw.
14. `(rear + 1) % MAX == front`.
15. **3, 4, 2** (LIFO); 1 remains.
16. Singly: `temp->next == NULL`. Circular: `temp->next == head`.
17. `temp` never moves, so the loop never ends (infinite loop).
18. After `delete head`, reading `head->next` uses freed memory; the rest of the list is lost or the program crashes. Save `temp = head`, move head, then delete temp.
19. In the circular list the last node must be re-pointed to the new head, and finding it means walking the whole list.
20. It's a variable-length array: not standard C++. Visual Studio rejects it and strict g++ errors on it. Use a fixed maximum like `int arr[100]`.
21. `(*p).data`.
22. Insertion sort: **stable**. Quick sort: **not stable**.